مؤسسه آموزش عالی اشراق (Eshragh Higher Education Institute)
دانشکده فنی و مهندسی — گروه مهندسی کامپیوتر

طراحی، پیاده‌سازی و تحلیل تجربی الگوریتم زمان‌بندی بی‌درنگ Maximum Urgency First (MUF) با استفاده از معماری‌های همگام‌سازی Semaphore، Monitor و ساختار ترکیبی (Hybrid)

عنوان فرعی: همگام‌سازی ناحیه بحرانی (Critical Section)، محافظت از سستی پویا (Dynamic Laxity)، مهار وارونگی اولویت (Priority Inversion)، مطالعات موردی ۳، ۵ و ۱۰ تسک و تحلیل زمان‌های بنیادی سیستم‌عامل

مشخصات شناسنامه‌ای پژوهش	جزئیات
مؤسسه آموزشی / دانشگاه	مؤسسه آموزش عالی اشراق (Eshragh Higher Education Institute)
گروه آموزشی	گروه مهندسی کامپیوتر (Department of Computer Engineering)
عنوان درس	سیستم‌های نهفته و بی‌درنگ (Embedded and Real-Time Systems) / سیستم‌عامل
پژوهشگر و تهیه‌کننده	امیرعلی تبریزنیا (Amirali Tabriznia)
استاد راهنما / مدرس درس	استاد مجید شمشیرگران (Majid Shamshirgaran)
تاریخ انتشار	مهر ۱۴۰۵ (October 2026)
الگوریتم زمان‌بندی محوری	الگوریتم دو-سطحی Maximum Urgency First (MUF) در مقایسه با RM و EDF
معماری‌های همگام‌سازی مورد مقایسه	۱. مبتنی بر سمافور (MUF-Sem) ۲ | مبتنی بر مانیتور (MUF-Mon) ۳ | ترکیبی (MUF-Hybrid)
موتور نرم‌افزاری شبیه‌ساز	muf_sync_benchmark.py (شبیه‌ساز دو-حالته کرنل بی‌درنگ و چندنخی در Python 3)

چکیده (Abstract)

الگوریتم زمان‌بندی Maximum Urgency First (MUF) که توسط Stewart و Khosla (۱۹۹۱) معرفی شد، تضاد کلاسیک میان هدررفت ظرفیت پردازنده در الگوریتم Rate Monotonic (RM) و فروپاشی دومینویی (Domino Effect) در الگوریتم Earliest Deadline First (EDF) هنگام اضافه‌بار گذرا را از طریق یک معماری اولویت دو-سطحی (Two-Tier) برطرف می‌سازد: سطح اول با اولویت ایستا، مجموعه تسک‌های بحرانی ($K_i = 1$) را ایزوله می‌کند و سطح دوم، تسک‌های هم‌سطح را به صورت پویا بر اساس بیشترین فوریت (کمترین سستی $L_i(t) = d_i - t - R_i(t)$) زمان‌بندی می‌نماید.

با این حال، هنگامی که تسک‌های بی‌درنگ برای ورود به نواحی بحرانی مشترک (Critical Sections) با یکدیگر رقابت می‌کنند، سازوکارهای همگام‌سازی کلاسیک می‌توانند تضمین‌های ریاضی MUF را مختل کنند. در این پژوهش، سه معماری همگام‌سازی برای الگوریتم MUF طراحی، پیاده‌سازی و ارزیابی شده‌اند: (۱) معماری مبتنی بر سمافور همراه با ارث‌بری پویای اولویت (۲) ،(MUF-Sem) معماری مبتنی بر مانیتور و متغیرهای شرطی (MUF-Mon)، و (۳) معماری ترکیبی سمافور-مانیتور (MUF-Hybrid). عملکرد این سه روش در سه مقیاس آزمایشی (۳، ۵ و ۱۰ تسک) از منظر زمان انتظار، توزیع دنباله انتظار، تورم مدت نگه‌داشتن قفل، هزینه CPU، تعداد Context Switch، انصاف و زمان‌های بنیادی سیستم‌عامل مورد واکاوی قرار گرفته است.

فهرست مطالب (ساختار ده فصلی گزارش)

عنوان فصل	زیربخش‌های پوشش داده‌شده	وضعیت
فصل ۱: مقدمه و بیان مسئله	۱-۱ مسئله ناحیه بحرانی و رقابت منابع در زمان‌بندی MUF ۱-۲ سه معماری همگام‌سازی مورد مقایسه (Semaphore, Monitor, Hybrid) ۱-۳ هدف و ساختار گزارش	کامل در متن
فصل ۲: مبانی ریاضی و معماری الگوریتم MUF	۲-۱ بردار اولویت دو-سطحی: بحرانیت ایستا (K_i) و سستی پویا ($L_i(t)$) ۲-۲ تشخیص زودهنگام خطای ددلاین ($L_i(t) < 0$) و قانون Sticky Priority ۲-۳ تأثیر مسدودشدن منابع (Blocking B_i) بر فوریت پویا $U_i(t)$	کامل در متن
فصل ۳: سمافور، مانیتور و معماری ترکیبی در MUF	۳-۱ پیاده‌سازی MUF با سمافور و ارث‌بری پویای اولویت (MUF-Sem + PIP) ۳-۲ پیاده‌سازی MUF با مانیتور Hoare/Mesa و صف شرطی (MUF-Mon) ۳-۳ معماری ترکیبی سمافور-مانیتور (MUF-Hybrid: دروازه SRP + سمافور سریع) ۳-۴ مقایسه مفهومی و ساختاری روش‌ها	کامل در متن
فصل ۴: مدل آزمایش و روش اندازه‌گیری	۴-۱ برنامه آزمایشی و معماری شبیه‌ساز دو-حالته در Python ۴-۲ معیارهای ارزیابی و تعاریف ریاضی ۴-۳ محدودیت‌های اجرای Python (قفل GIL، زمان‌بندی نخ‌ها و راهکارها)	کامل در متن
فصل ۵: مثال اول با ۳ تسک (رقابت سبک)	۵-۱ مشخصات مسئله و پروفایل ناحیه بحرانی (۳ تسک) ۵-۲ نتایج تجربی، ردیابی گام‌به‌گام و مقایسه نمودار گانت	کامل در متن
فصل ۶: مثال دوم با ۵ تسک (اضافه‌بار و تله وارونگی)	**۶-۱ مشخصات مسئله و مخاطره انسداد چندمنبعی (۵ تسک)** **۶-۲ نتایج تجربی: شکست زنجیره‌ای PIP در برابر حفاظت MUF-Hybrid**	کامل در متن
فصل ۷: مثال سوم با ۱۰ تسک (آزمون استرس رقابت بالا)	۷-۱ مشخصات مسئله و رقابت چندسطحی روی ۳ منبع (۱۰ تسک) ۷-۲ نتایج تجربی: طوفان بیدارسازی Mesa در برابر مقیاس‌پذیری Hybrid	کامل در متن
فصل ۸: تحلیل تطبیقی نتایج در مقیاس‌های مختلف	۸-۱ زمان انتظار | ۸-۲ دنباله توزیع انتظار (P50, P95, P99) ۸-۳ گذردهی و تورم مدت نگه‌داشتن | ۸-۴ هزینه CPU و تأخیر بدون رقابت ۸-۵ تعویض متن ۸-۶ | (Context Switch) انصاف و انتظار محدود ۸-۷ تحلیل رفتار نخ‌ها در اضافه‌بار گذرا	کامل در متن
فصل ۹: مقایسه از دیدگاه زمان‌های مهم سیستم‌عامل	۹-۱ تجزیه زمان‌های بنیادی (RT, WT_ready, B_i, TAT) ۹-۲ تفسیر سیستمی: موازنه زمان پاسخ اولیه در برابر زمان گردش کل	کامل در متن
فصل ۱۰: تحلیل نهایی و جمع‌بندی + مراجع	۱۰-۱ خلاصه یافته‌ها | ۱۰-۲ تحلیل و سنتز جامع ۱۰-۳ توصیه‌های کاربردی | ۱۰-۴ محدودیت‌ها و توسعه چند-هسته‌ای (CP-MUF) ۱۰-۵ نتیجه‌گیری نهایی و مراجع (References)	کامل در متن

فصل ۱: مقدمه و بیان مسئله

در سیستم‌های نهفته و بی‌درنگ سخت (Hard Real-Time Embedded Systems)—از عملگرهای کنترل پرواز (Fly-by-Wire) و ترمز ضدقفل خودرو (ABS) گرفته تا رباتیک جراحی و کنترل فرآیندهای صنعتی—صحت عملکرد سیستم صرفاً به درستی منطقی محاسبات وابسته نیست، بلکه نیازمند قطعیت زمانی (Temporal Determinism) است؛ به این معنا که هر تسک بحرانی باید پیش از انقضای ددلاین فیزیکی خود (d_i) به پایان برسد. نظریه کلاسیک زمان‌بندی بی‌درنگ همواره با یک دوگانگی بنیادی روبه‌رو بوده است: الگوریتم اولویت ایستای Rate Monotonic (RM) در شرایط اضافه‌بار رفتار قابل‌پیش‌بینی دارد اما کران بهره‌وری پردازنده آن به U = n(2^(1/n) - 1) ≈ 69.31% محدود شده و تا ۳۰.۷٪ ظرفیت CPU را بلااستفاده می‌گذارد؛ در مقابل، الگوریتم اولویت پویای Earliest Deadline First (EDF) در بار عادی به بهره‌وری ۱۰۰٪ دست می‌یابد، اما به محض بروز اضافه‌بار گذرا (U > 100%) دچار فروپاشی زنجیره‌ای یا اثر دومینو (Domino Effect) شده و باعث از دست رفتن ددلاین تمامی تسک‌های بحرانی می‌شود.

برای حل این چالش تاریخی، David B. Stewart و Pradeep K. Khosla در سال ۱۹۹۱ در دانشگاه کارنگی ملون الگوریتم زمان‌بندی Maximum Urgency First (MUF) را برای سیستم‌عامل بی‌درنگ Chimera II طراحی کردند. الگوریتم MUF با ترکیب جداسازی ایستای بحرانیت در سطح اول (Tier 1: K_i) و زمان‌بندی پویا بر پایه کمترین سستی یا بیشترین فوریت در سطح دوم (Tier 2: L_i(t) = d_i - t - R_i(t))، هم بهره‌وری ۱۰۰٪ را برای مجموعه تسک‌های بحرانی تضمین می‌کند و هم در برابر اثر دومینو ایمن است. با این حال، در کاربردهای واقعی، تسک‌ها به صورت مستقل اجرا نمی‌شوند بلکه برای دسترسی به منابع مشترک (مانند گذرگاه‌های سخت‌افزاری، بافرهای حافظه و ثبات‌های عملگر) وارد نواحی بحرانی (Critical Sections) می‌شوند که همگام‌سازی نادرست آن‌ها می‌تواند تضمین‌های ریاضی MUF را به کلی از بین ببرد.

۱-۱ مسئله ناحیه بحرانی و رقابت منابع در زمان‌بندی MUF

فرض کنید کرنل یک سیستم‌عامل بی‌درنگ مجموعه‌ای از n تسک هم‌روند $\Gamma = \{P_1, P_2, ..., P_n\}$ را اجرا می‌کند. در هر تیک ساعت t، هر تسک P_i با زمان ورود A_i، زمان اجرای بدترین حالت C_i، زمان اجرای باقی‌مانده $R_i(t)$، ددلاین مطلق $d_i = A_i + D_i$ و سطح بحرانیت ایستا $K_i \in \{0, 1\}$ مشخص می‌شود. در غیاب رقابت منابع، زمان‌بند MUF تسکی را انتخاب می‌کند که بردار اولویت زیر را بیشینه کند:

معادلات بردار اولویت دو-سطحی و سستی پویا در الگوریتم MUF تحت رقابت منابع

$$L_i(t) = d_i - t - R_i(t) \quad \text{[Dynamic Laxity / Slack Time at tick t]}$$

$$U_i(t) = -L_i(t) = t + R_i(t) - d_i \quad \text{[Dynamic Urgency at tick t]}$$

$$\text{MUF_Priority}(P_i, t) = (K_i, U_i(t), \text{Sticky}_i, -A_i, -\text{PID}_i)$$

$$L_i^{eff}(t) = d_i - t - R_i(t) - B_i(t) \quad \text{[Effective Laxity degraded by Blocking } B_i(t)\text{]}$$

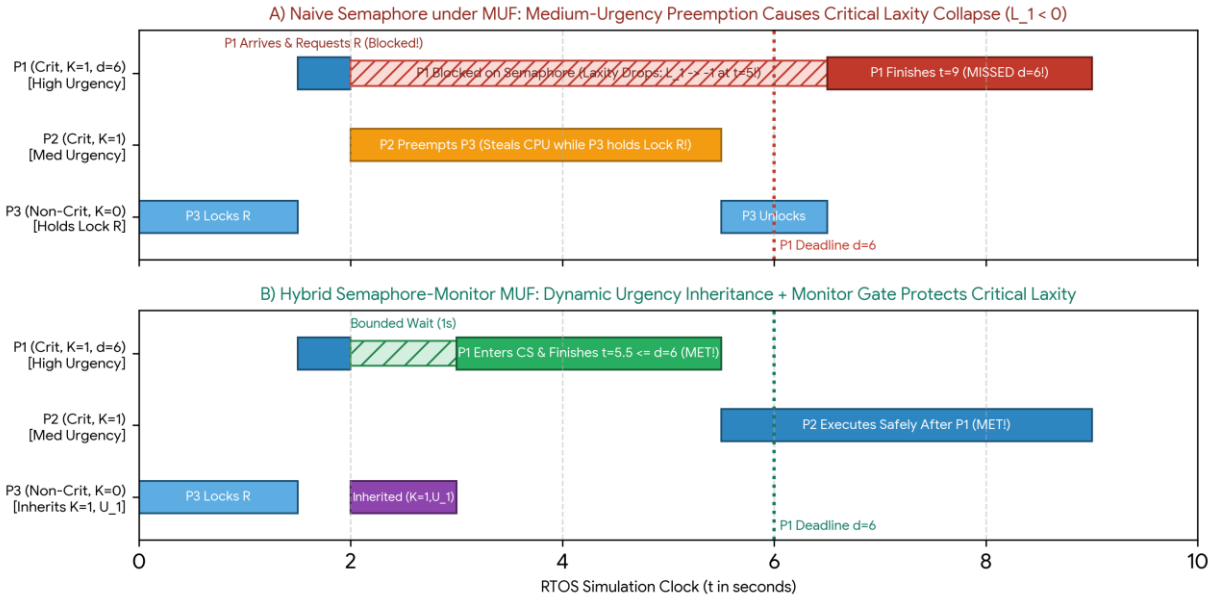
هنگامی که تسک‌های هم‌روند برای ورود به ناحیه بحرانی (Critical Section) نیازمند اخذ قفل انحصاری (Mutual Exclusion) هستند، انسداد (Blocking) در زمان‌بند MUF چهار ناهنجاری زمانی خطرناک ایجاد می‌کند:

۱. فرسایش مستقیم سستی پویا در حالت مسدود (Dynamic Laxity Erosion): زمانی که یک تسک روی CPU اجرا می‌شود، به ازای هر تیک پیشرفت زمان t، زمان باقی‌مانده اجرای آن $R_i(t)$ نیز یک واحد کاهش می‌یابد و در نتیجه سستی آن ثابت می‌ماند ($\Delta L_{run} / \Delta t = 0$). اما هنگامی که تسک P_i پشت قفل یک Semaphore یا Monitor مسدود می‌شود، $R_i(t)$ ثابت مانده در حالی که ساعت t جلو می‌رود؛ بنابراین هر تیک انسداد $B_i(t)$ مستقیماً سستی تسک را با نرخ $\Delta L_i = -\Delta t$ کاهش داده و آن را به سمت سستی منفی ($L_i(t) < 0$) و شکست ددلاین سوق می‌دهد.

۲. وارونگی اولویت بین-سطحی و درون-سطحی (Priority Inversion): اگر یک تسک غیربحرانی P_low (K=0) قفل ناحیه بحرانی را در اختیار بگیرد و سپس توسط یک تسک بحرانی با فوریت متوسط P_med (K=1) که نیازی به آن قفل ندارد قبضه (Preempt) شود، تسک بحرانی با فوریت بالا P_high (K=1) که متقاضی همان قفل است، پشت هر دو تسک P_low و P_med مسدود می‌ماند و جداسازی سطح اول MUF کاملاً نقض می‌گردد.

۳. کهنگی ترتیب فوریت در صف‌های انتظار قفل (Stale Queue Ordering): صف‌های انتظار استاندارد FIFO در سمافورها و مانیتورها، نخ‌ها را بر اساس زمان ورود مرتب می‌کنند؛ در حالی که در MUF، فوریت $U_i(t)$ هر نخ مسدود در هر تیک ساعت افزایش می‌یابد و صف انتظار باید در لحظه آزادسازی قفل بر اساس آخرین وضعیت $(K_i, U_i(t))$ بازآرایی شود.

۴. تورم مدت نگه‌داشتن قفل (Hold-Time Inflation): قبضه‌شدن نخی که در درون ناحیه بحرانی قرار دارد باعث می‌شود مدت زمان بسته ماندن قفل از زمان خالص C_{cs} به $C_{cs} + \text{Preemption_Delay}$ افزایش یافته و اثر کاروانی (Convoy Effect) برای سایر نخ‌ها ایجاد کند.



شکل ۱-۱: مقایسه رفتار زمانی (A) سمافور ساده در MUF که در آن قبضه‌شدن نخ دارنده قفل P3 توسط P2 باعث منفی شدن سستی تسک بحرانی P1 و شکست ددلاین آن می‌شود، در برابر (B) معماری ترکیبی MUF-Hybrid با ارث‌بری پویای فوریت.

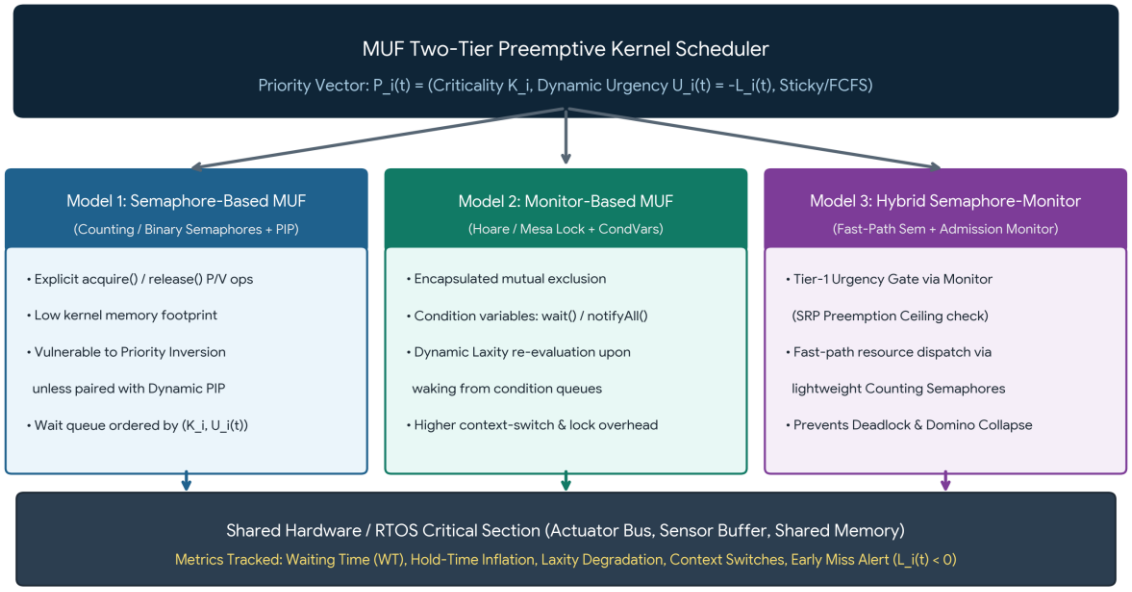
۱-۲ سه معماری همگام‌سازی مورد مقایسه (Semaphore, Monitor, Hybrid)

برخلاف الگوریتم‌های انتظار مشغول (Busy-Waiting / Spinlocks) نظیر الگوریتم‌های Peterson و Dekker که چرخه‌های پردازنده تک‌هسته‌ای را هدر داده و با قبضه‌کردن پویای MUF ناسازگارند، سیستم‌عامل‌های بی‌درنگ از سازوکارهای مسدودکننده کرنل استفاده می‌کنند. در این گزارش، سه معماری همگام‌سازی اختصاصی برای الگوریتم MUF طراحی و با یکدیگر مقایسه شده‌اند:

معماری اول – پیاده‌سازی MUF با سمافور و ارث‌بری پویا (MUF-Sem): مبتنی بر سمافورهای شمارشی و دودویی (عملیات P/acquire و V/release) مجهز به هرم انتظار مرتب‌شده با بردار ((K_i, U_i(t)) و پروتکل ارث‌بری پویای اولویت (Dynamic PIP). در این روش، هرگاه تسک بحرانی P_i پشت سمافور متعلق به P_owner مسدود شود، P_owner فوراً زوج (max(K), max(U(t))) را به ارث می‌برد. این روش در حالت بدون رقابت بسیار سریع است اما در برابر انسداد زنجیره‌ای (Chained Blocking) روی چند منبع و بن‌بست (Deadlock) آسیب‌پذیر است.

معماری دوم – پیاده‌سازی MUF با مانیتور و متغیرهای شرطی (MUF-Mon): داده‌های مشترک و رویه‌های ناحیه بحرانی را درون ساختار کپسوله‌شده Monitor به همراه قفل انحصاری درونی و متغیرهای شرطی (wait, notify, notifyAll) قرار می‌دهد. هنگام فراخوانی notifyAll()، مانیتور سستی لحظه‌ای تمامی نخ‌های منتظر را بازمحاسبه کرده و نخ دارای بیشترین ((K_i, U_i(t)) را بیدار می‌کند. در این پژوهش، تفاوت سیگنالینگ Mesa (رقابت مجدد نخ بیدارشده برای قفل مانیتور) و Hoare (تحویل مستقیم قفل) تحت زمان‌بند MUF تحلیل می‌شود.

معماری سوم — معماری ترکیبی سمافور و مانیتور (MUF-Hybrid): یک معماری نوآورانه دو-مرحله‌ای منطبق بر ساختار دو-سطحی MUF. در مرحله اول (Stage 1)، یک دروازه پذیرش مانیتور (Monitor Admission Gate) با اعمال سقف پیش‌گیری Stack Resource Policy (SRP) و فیلتر زودهنگام سستی منفی (رد درخواست تسک‌های غیربحرانی $K=0$ که $L_i(t) < 0$ دارند پیش از قفل کردن منبع) ورود تسک‌ها را کنترل می‌کند. در مرحله دوم (Stage 2)، تخصیص واحدهای منبع از طریق یک سمافور شمارشی مسیر سریع (Fast-Path Semaphore) بدون هیچ‌گونه انسداد ثانویه یا طوفان بیدارسازی انجام می‌گیرد.



شکل ۱-۲: معماری کلی سه روش همگام‌سازی مورد ارزیابی تحت زمان‌بند دو-سطحی (۱) MUF: سمافور (۲) MUF-Sem، مانیتور MUF-Mon، و (۳) ساختار ترکیبی MUF-Hybrid.

ویژگی معماری	۱. روش سمافور (MUF-Sem)	۲. روش مانیتور (MUF-Mon)	۳. روش ترکیبی (MUF-Hybrid)
سازوکار پایه همگام‌سازی	سمافور دودویی/شمارشی (P/V) + پروتکل Dynamic PIP	قفل کپسوله‌شده مانیتور + متغیرهای شرطی (CondVar)	**دو-مرحله‌ای: دروازه مانیتور SRP + سمافور مسیر سریع**
سیاست مرتب‌سازی صف انتظار	هرم اولویت (Min-Laxity Heap) با بازآرایی در لحظه آزادسازی	ارزیابی زنده ((K_i, U_i(t)) هنگام فراخوانی notifyAll())	**غربالگری پیش‌گیرانه با سقف SRP + اخذ O(1) سمافور**
تأخیر اخذ قفل بدون رقابت	کمترین (کاهش اتمیک شمارنده سمافور)	متوسط تا زیاد (قفل مانیتور + بررسی شرط حلقه while)	**کم (بررسی اسکالر سقف SRP + کاهش سریع سمافور)**
مقابله با وارونگی اولویت	واکنشی (Reactive — فعال‌سازی PIP پس از وقوع انسداد)	واکنشی / وابسته به مدل سیگنالینگ Mesa یا Hoare	**پیش‌گیرانه (Proactive — جلوگیری از انسداد پیش از ورود)**
حفاظت در برابر تسک‌های محکوم به شکست (L_i < 0)	ندارد (تسک غیربحرانی با L<0 همچنان می‌تواند قفل را بگیرد)	جزئی (نیازمند بررسی دستی گزاره درون حلقه شرط)	**کامل (دروازه مانیتور تسک‌های K=0 با L_i<0 را رد می‌کند)**

۱-۳ هدف و ساختار گزارش

هدف اصلی این پژوهش، ایجاد پیوند میان نظریه زمان‌بندی بی‌درنگ MUF و مکانیزم‌های همگام‌سازی سیستم‌عامل از طریق فرمول‌بندی ریاضی، شبیه‌سازی کرنل و اجرای بنچمارک‌های چندنخی در پایتون است. در ادامه، فصل ۲ مبانی ریاضی سستی پویا و نرخ فرسایش آن در حالت مسدود را تبیین می‌کند؛ فصل ۳ طراحی دقیق و شبه-کدهای هر سه روش MUF-Mon، MUF-Sem و MUF-Hybrid را ارائه می‌دهد؛ فصل ۴ مدل آزمایشی و محدودیت‌های پایتون (GIL) را بررسی می‌کند؛ فصول ۵، ۶ و ۷ مطالعات موردی ۳، ۵ و ۱۰ تسک را اجرا می‌کنند؛ فصل ۸ تحلیل تطبیقی ۷ معیار عملکردی را ارائه می‌دهد؛ فصل ۹ نتایج را از دیدگاه زمان‌های بنیادی سیستم‌عامل کالبدشکافی می‌کند و فصل ۱۰ جمع‌بندی و توصیه‌های طراحی را ارائه می‌نماید.

فصل ۲: مبانی ریاضی و معماری الگوریتم MUF

برای طراحی سازوکارهای همگام‌سازی قطعی با استفاده از سمافورها، مانیتورها و دروازه‌های ترکیبی، ابتدا باید رفتار ریاضی زمان‌بند Maximum Urgency First (MUF) را به صورت دقیق فرموله کنیم. برخلاف زمان‌بندهای اولویت ایستا (نظیر RM) یا زمان‌بندهای تک‌بعدی پویا (نظیر EDF)، الگوریتم MUF در هر رویداد کرنل (تیک ساعت، ورود تسک جدید، درخواست قفل یا آزادسازی قفل) یک بردار اولویت چندبعدی را ارزیابی می‌کند.

۲-۱ بردار اولویت دو-سطحی: بحرانیت ایستا (K_i) و سستی پویا L_i(t)

فرض کنید $\Gamma = \{P_1, P_2, ..., P_n\}$ مجموعه‌ای از n تسک بی‌درنگ با قابلیت قبضه‌شدن (Preemptive) باشد. هر تسک P_i با پنج‌تایی $P_i = (A_i, C_i, D_i, T_i, K_i)$ تعریف می‌شود که در آن A_i زمان ورود، C_i بدترین زمان اجرای پردازنده D_i (WCET)، $d_i = A_i + D_i$ ددلاین نسبی، T_i دوره تناوب و $K_i \in \{0, 1\}$ سطح بحرانیت ایستا است. در مرحله کنترل پذیرش آفلاین، تسک‌ها به دو زیرمجموعه مجزا افراز می‌شوند:

افراز آفلاین مجموعه تسک‌های بحرانی (Critical Set) و کران بهره‌وری در MUF

$$\Gamma_{crit} = \{ P_i \in \Gamma \mid K_i = 1 \} \quad \text{such that} \quad U_{crit} = \sum_{P_i \in \Gamma_{crit}} (C_i / T_i) <= 1.00 \ (100\%)$$

$$\Gamma_{non} = \{ P_i \in \Gamma \mid K_i = 0 \} \quad \text{where} \quad U_{total} = U_{crit} + U_{non} \text{ may exceed } 1.00 \text{ during overload}$$

در لحظه t از اجرای سیستم، اگر $R_i(t) \in [0, C_i]$ زمان اجرای باقی‌مانده تسک P_i باشد، سستی پویا (Dynamic Laxity یا Slack Time) برابر با بیشترین مدتی است که تسک P_i می‌تواند به تأخیر بیفتد یا پشت قفل مسدود شود بدون آنکه ددلاین مطلق d_i خود را از دست بدهد. فوریت پویا $U_i(t)$ قرینه سستی پویا تعریف می‌شود:

معادلات سستی پویا (Laxity)، فوریت (Urgency) و بردار اولویت لغت‌نامه‌ای MUF

$$L_i(t) = d_i - t - R_i(t) \quad \text{[Remaining Slack before Deadline Miss]}$$

$$U_i(t) = -L_i(t) = t + R_i(t) - d_i \quad \text{[Dynamic Urgency — Monotonically increases during wait]}$$

$$Sticky_i(t) = 1 \text{ if } (P_i == Running_Task) \text{ else } 0 \quad \text{[Anti-Thrashing Sticky Execution Bit]}$$

$$Priority_MUF(P_i, t) = (K_i, U_i(t), Sticky_i(t), -A_i, -PID_i)$$

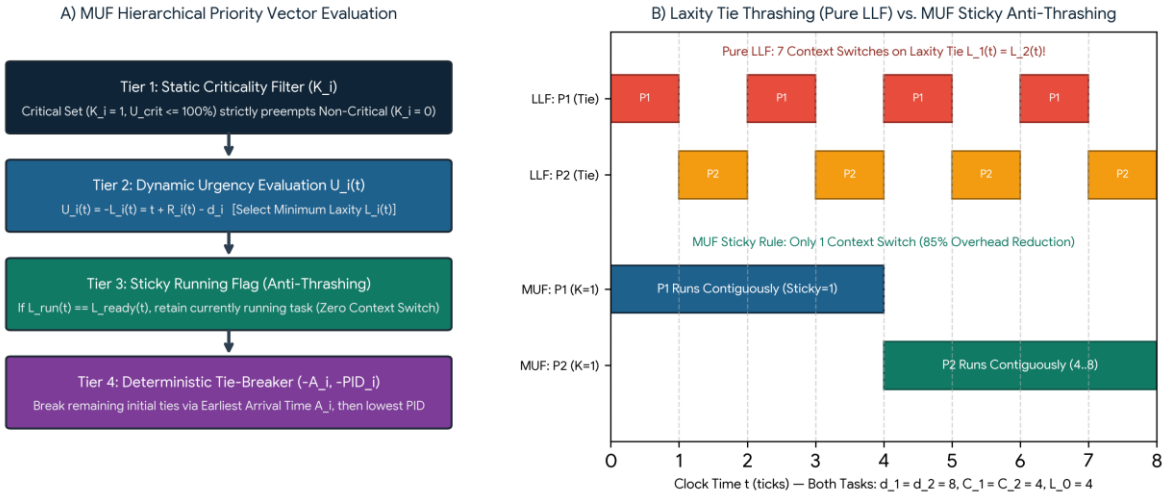
هنگام مقایسه دو تسک آماده P_a و P_b در لحظه t، توزیع‌کننده کرنل مقایسه لغت‌نامه‌ای (Lexicographical) $Priority_MUF(P_a, t) > Priority_MUF(P_b, t)$ را اجرا می‌کند: ابتدا سطح اول یعنی بحرانیت ایستا ($K_a > K_b$) بررسی می‌شود؛ در صورت برابری سطح بحرانیت، سطح دوم یعنی فوریت پویا ($U_a(t) > U_b(t)$) یا معادلاً سستی کمتر ($L_a(t) < L_b(t)$) ملاک قرار می‌گیرد؛ و در صورت تساوی سستی، بیت چسبندگی تسک در حال اجرا ($Sticky_i$) و زمان ورود زودتر ($-A_i$) بررسی می‌شوند.

۲-۲ تشخیص زودهنگام خطای ددلاین ($L_i(t) < 0$) و قانون Sticky ضد-Thrashing

دو ویژگی معماری منحصربه‌فرد، الگوریتم MUF را در مدیریت نخ‌های هم‌روند و منابع مشترک از RM و EDF متمایز می‌سازد:

۱. تشخیص زودهنگام شکست ددلاین (Early Deadline Failure Detection): در الگوریتم‌های RM و EDF، کرنل تا لحظه رسیدن ساعت به t = d_i متوجه از دست رفتن ددلاین نمی‌شود؛ در نتیجه تسکی که محکوم به شکست است همچنان CPU و قفل‌های سمافور/مانیتور را اشغال می‌کند. در مقابل، در الگوریتم MUF به محض آنکه سستی یک تسک منفی شود (L_i(t) < 0)، از نظر ریاضی قطعی است که آن تسک حتی با اختصاص ۱۰۰٪ پردازنده نیز به ددلاین نخواهد رسید. این خطا در لحظه t_alert = d_i - R_i(t) + 1 (مدت‌ها پیش از d_i) کشف شده و به کرنل اجازه می‌دهد قفل‌های در اختیار تسک غیربحرانی محکوم به شکست را فوراً آزاد کند.

۲. مهار پدیده Laxity Tie Thrashing با قانون Sticky Priority: در الگوریتم خالص Least Laxity First (LLF)، زمانی که دو تسک هم‌سطح به سستی برابر L_1(t) = L_2(t) می‌رسند، اجرای P_1 به مدت ۱ تیک باعث ثابت ماندن L_1 و کاهش L_2 به اندازه ۱ واحد می‌شود؛ در تیک بعدی P_2 تسک P_1 را قبضه کرده و این تساوی در هر تیک تکرار می‌شود که به وقوع Context Switch در تک‌تک تیک‌های ساعت می‌انجامد! الگوریتم MUF با قرار دادن Sticky_i = 1 برای تسک در حال اجرا، تضمین می‌کند که در صورت تساوی سستی، تسک جاری پردازنده را حفظ کرده و از تعویض متن‌های بیهوده جلوگیری شود.



شکل ۲-۱: (A) خط لوله چهارسطحی ارزیابی بردار اولویت در MUF؛ (B) مقایسه پدیده Laxity Tie Thrashing در LLF خالص (۷ تعویض متن) در برابر قانون Sticky Priority در MUF (تنها ۱ تعویض متن و کاهش ۸۵٪ سربار).

۲-۳ تأثیر مسدودشدن منابع (Blocking B_i) بر فوریت پویا U_i(t)

نرخ تغییرات سستی پویا (dL_i / dt) و فوریت پویا (dU_i / dt) در سه وضعیت اصلی نخ در سیستم‌عامل (در حال اجرا روی CPU، آماده در صف Ready، و مسدود پشت قفل Semaphore یا Monitor) از معادلات دیفرانسیل زیر پیروی می‌کند:

معادلات دیفرانسیل تغییرات سستی و فوریت در وضعیت‌های سه‌گانه نخ و شرط زمان‌بندی‌پذیری با انسداد B_i

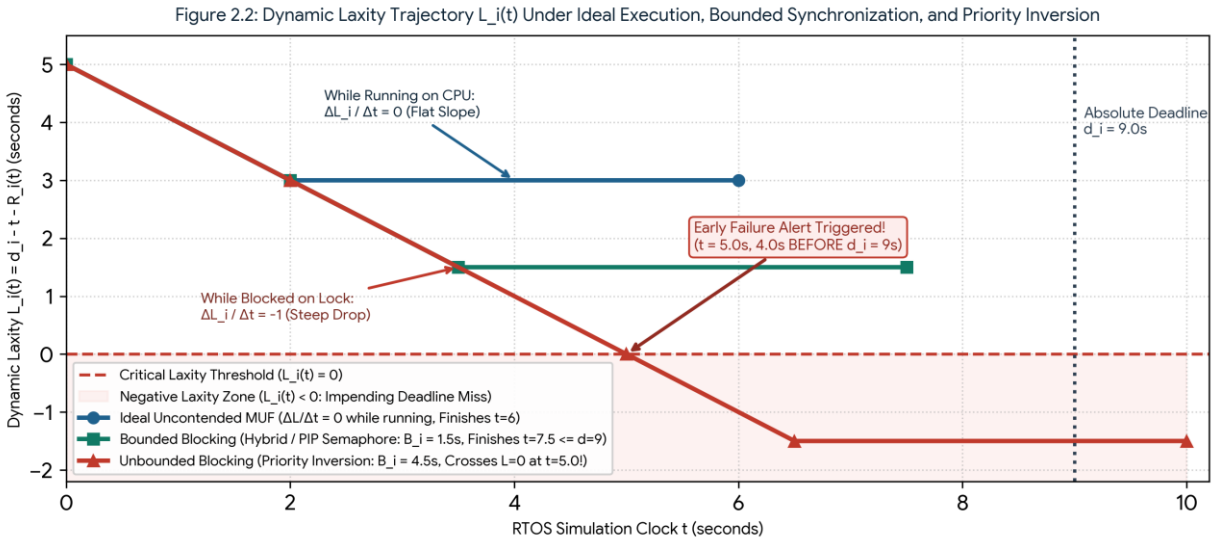
State 1 (Running on CPU): dR_i/dt = -1 ==> dL_i/dt = 0, dU_i/dt = 0 (Constant Laxity)

State 2 (Ready Queue Wait): dR_i/dt = 0 ==> dL_i/dt = -1, dU_i/dt = +1 (Anti-Starvation)

State 3 (Blocked on Sem/Mon): dR_i/dt = 0 ==> dL_i/dt = -1, dU_i/dt = +1 (Unschedulable Slack Loss!)

Schedulability Condition: C_i + I_crit(P_i) + B_i <= D_i <==> B_i <= L_i(A_i) - I_crit(P_i)

همان‌گونه که معادلات بالا نشان می‌دهند، اگرچه در هر دو وضعیت Ready و Blocked نرخ کاهش سستی برابر با -1 واحد در هر تیک است، اما در وضعیت Ready افزایش فوریت U_i(t) باعث صعود خودکار نخ در صف آماده CPU شده و مانع از گرسنگی درون-سطحی (Intra-Tier Starvation) می‌شود؛ در حالی که در وضعیت Blocked نخ از صف آماده CPU خارج شده است و تا زمانی که مالک قفل (P_owner) ناحیه بحرانی را آزاد نکند، افزایش فوریت هیچ کمکی به اجرای نخ مسدود نمی‌کند.



شکل ۲-۲: مسیر تغییرات سستی پویا L_i(t) در سه حالت اجرای بدون رقابت (شیب صفر حین اجرا)، همگام‌سازی کران‌دار (MUF-Hybrid / PIP) و وارونگی اولویت نامحدود (عبور از مرز بحرانی L=0 در لحظه t=5.0s).

وضعیت نخ در سیستم‌عامل	تغییرات R_i(t)	نرخ سستی (dL_i/dt)	نرخ فوریت (dU_i/dt)	الزام همگام‌سازی روی سمافور و مانیتور
در حال اجرا (عادی)	کاهش (-1 / tick)	0 (ثابت)	0 (ثابت)	حفظ بیت Sticky_i = 1 برای جلوگیری از Thrashing در تساوی سستی.
در حال اجرا (درون ناحیه بحرانی)	کاهش (-1 / tick)	0 (ثابت)	0 (ثابت)	الزام به ارث‌بری max(K_wait, U_wait) یا سقف SRP جهت جلوگیری از تورم قفل.
آماده در صف پردازنده (Ready)	ثابت (0 / tick)	-1 / tick (کاهشی)	+1 / tick (افزایشی)	افزایش خودکار اولویت در صف آماده MUF؛ تضمین عدم گرسنگی درون-سطحی.
مسدود پشت قفل (Blocked)	ثابت (0 / tick)	-1 / tick (فرسایش!)	+1 / tick (افزایشی!)	صف انتظار قفل باید بر اساس (K_i, U_i(t)) زنده مرتب شده و فوریت را به مالک قفل منتقل کند.

فصل ۳: سمافور، مانیتور و معماری ترکیبی در MUF

در این فصل، طراحی معماری و الگوریتم‌های سطح کرنل برای سه روش همگام‌سازی ناحیه بحرانی در زمان‌بند MUF ارائه می‌شود: (۱) سمافور هوشمند مجهز به ارث‌بری پویای اولویت (۲) ،(MUF-Sem) مانیتور مجهز به صف شرطی حساس به سستی لحظه‌ای (MUF-Mon)، و (۳) معماری ترکیبی دو-مرحله‌ای سمافور و مانیتور (MUF-Hybrid).

۳-۱ پیاده‌سازی MUF با سمافور و ارث‌بری پویای اولویت (MUF-Sem)

سمافور کلاسیک دایکسترا شامل یک شمارنده صحیح S.count و یک صف انتظار S.wait_queue است که با دو دستور اتمیک P(S) (یا acquire) و V(S) (یا release) مدیریت می‌شود. برای سازگاری کامل سمافور با الگوریتم MUF، دو تغییر بنیادی در ساختار آن اعمال کرده‌ایم: نخست، تبدیل صف انتظار FIFO به یک هرم اولویت پویا (Urgency Wait Heap) که نخ‌های مسدود را بر اساس بردار (K_i, U_i(t), -A_i, -PID_i) مرتب می‌کند؛ و دوم، پیاده‌سازی موتور ارث‌بری پویای متعدی (Transitive Dynamic PIP) که به محض مسدود شدن نخ P_curr پشت سمافور S، بحرانیت و فوریت مؤثر مالک فعلی قفل (P_owner) را به بیشینه مقادیر نخ‌های منتظر ارتقا می‌دهد:

الگوریتم ۳-۱: شبه-کد سمافور مبتنی بر MUF همراه با ارث‌بری پویای اولویت (MUF-Sem + Dynamic PIP)

```
PROCEDURE MUF_Sem_Acquire(Semaphore S, Task P_curr, Clock t):
    IF S.count > 0 THEN
        S.count <- S.count - 1;  S.owners.ADD(P_curr)
    ELSE
        P_curr.state <- BLOCKED_ON_SEM;  S.wait_queue.INSERT(P_curr)
        FOR EACH P_own IN S.owners DO  # Transitive Dynamic Urgency Inheritance
            P_own.K_eff <- MAX(P_own.K_eff, P_curr.K_i)
            P_own.U_eff(t) <- MAX(P_own.U_eff(t), P_curr.U_i(t))
        END FOR
        Invoke_MUF_Dispatcher(t)

PROCEDURE MUF_Sem_Release(Semaphore S, Task P_curr, Clock t):
    S.owners.REMOVE(P_curr);  Restore_Native_MUF_Priority(P_curr, t)
    IF S.wait_queue IS NOT EMPTY THEN
        Recalculate_Live_Laxity_All(S.wait_queue, t)  # O(k) live update
        P_next <- EXTRACT_MAX_MUF(S.wait_queue)       # Max (K_i, U_i(t))
        S.owners.ADD(P_next);  P_next.state <- READY
        Invoke_MUF_Dispatcher(t)
    ELSE  S.count <- S.count + 1  END IF
```

۳-۲ پیاده‌سازی MUF با مانیتور و متغیرهای شرطی (MUF-Mon)

مانیتورها با کپسوله‌سازی متغیرهای حالت مشترک، قفل انحصاری درونی (M.mutex) و متغیرهای شرطی (Condition Variables) درون یک کلاس واحد، خطاهای برنامه‌نویسی ناشی از فراخوانی‌های پراکنده P/V در سمافورها را برطرف می‌کنند. در معماری MUF-Mon، هرگاه نخی شرط ورود به ناحیه بحرانی را برقرار نبیند، دستور cond_free.wait() را فراخوانی کرده و قفل مانیتور را آزاد می‌کند. هنگام خروج مالک فعلی و اجرای notifyAll()، توزیع‌کننده درونی مانیتور سستی لحظه‌ای $L_j(t) = d_j - t - R_j(t)$ تمامی نخ‌های خفته در صف شرط را بازمحاسبه می‌کند. نکته کلیدی در MUF-Mon، تفاوت میان سیگنالینگ Mesa (که در آن نخ بیدارشده باید مجدداً برای اخذ قفل مانیتور با نخ‌های تازه وارد رقابت کند و مستعد پدیده Barging و تعویض متن‌های اضافی است) و سیگنالینگ Hoare (تحویل مستقیم قفل به فوری‌ترین نخ بیدارشده) است.

الگوریتم ۳-۲: شبه-کد مانیتور مبتنی بر MUF با بازمحاسبه سستی لحظه‌ای در متغیر شرطی (MUF-Mon)

```
MONITOR MUF_Resource_Monitor:
Boolean busy <- FALSE;  ConditionVariable cond_free

PROCEDURE Enter_Critical_Section(Task P_curr, Clock t):
Acquire_Monitor_Lock_With_PIP(P_curr, t)
WHILE busy == TRUE DO
Wait_Urgency_Ordered(cond_free, P_curr, t)
END WHILE
busy <- TRUE;  Release_Monitor_Lock(P_curr, t)

PROCEDURE Exit_Critical_Section(Task P_curr, Clock t):
Acquire_Monitor_Lock_With_PIP(P_curr, t)
busy <- FALSE
Notify_Most_Urgent_Or_All(cond_free, t)
Release_Monitor_Lock(P_curr, t)
```

۳-۳ معماری ترکیبی سمافور و مانیتور (MUF-Hybrid)

بررسی دقیق MUF-Sem و MUF-Mon نشان می‌دهد که این دو روش نقاط قوت و ضعف مکمل دارند: سمافور در مسیر بدون رقابت بسیار کم‌هزینه است اما در حضور چند منبع دچار انسداد زنجیره‌ای (Chained Blocking) و خطر بن‌بست شده و نمی‌تواند تسک‌های غیربحرانی محکوم به شکست ($L_i < 0$) را فیلتر کند؛ از سوی دیگر، مانیتور قابلیت بررسی گزاره‌های منطقی و فیلتر کردن وضعیت را دارد اما سیگنالینگ متغیرهای شرطی آن در رقابت

بالا سربار سنگینی تحمیل می‌کند. برای ترکیب مزایای هر دو، ما معماری دو-مرحله‌ای MUF-Hybrid را طراحی کرده‌ایم:

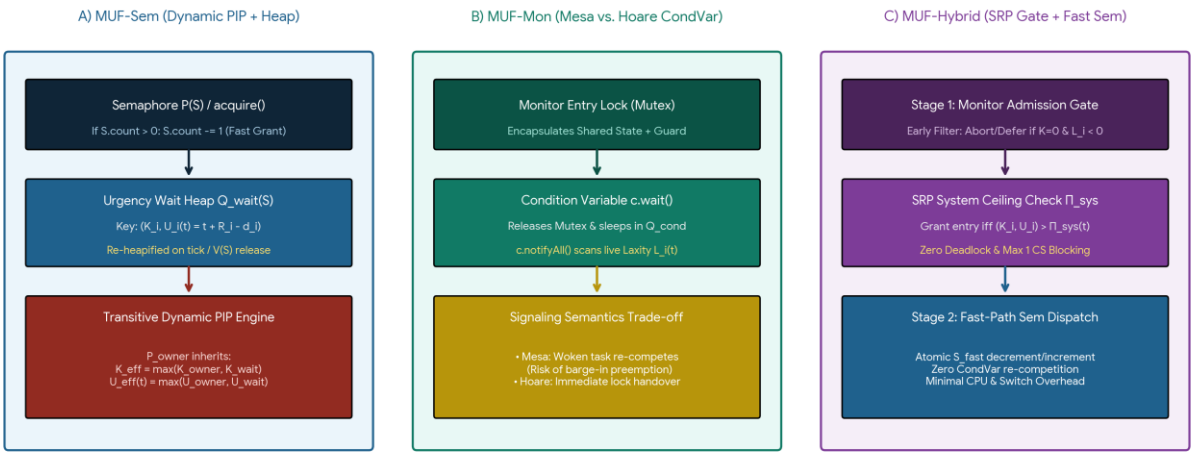
مرحله اول (Stage 1 — دروازه پذیرش مانیتور با سقف SRP و فیلتر زودهنگام سستی): پیش از آنکه نخ به سمافور منبع دست بزند، وارد یک دروازه سبک مانیتوری می‌شود. این دروازه ابتدا شرط سلامت سستی را بررسی می‌کند: اگر نخ متقاضی غیربحرانی باشد ($K_i = 0$) و سستی آن منفی شده باشد ($L_i(t) < 0$)، دروازه مانیتور فوراً درخواست را رد یا معلق می‌کند تا یک تسک محکوم به شکست هرگز منبعی را که تسک‌های بحرانی نیاز دارند قفل نکند! سپس، دروازه شرط سقف پیش‌گیری Stack Resource Policy (SRP) یعنی $Priority_MUF(P_{curr}, t) > \Pi_{sys}(t)$ را بررسی می‌کند تا تضمین شود تمام منابع مورد نیاز نخ آزاد بوده و هیچ انسداد متقاطعی رخ نخواهد داد.

مرحله دوم (Stage 2 — تخصیص منبع با سمافور شمارشی مسیر سریع): از آنجا که عبور از دروازه مانیتور در مرحله اول تضمین می‌کند که منبع آزاد است و نخ بالاتر از سقف پیش‌گیری سیستم قرار دارد، مرحله دوم صرفاً یک عملیات کاهش اتمیک روی سمافور مسیر سریع (Fast-Path Semaphore) انجام می‌دهد. هیچ نخی در مرحله دوم مسدود نمی‌شود و طوفان بیدارسازی Mesa، انسداد زنجیره‌ای و بن‌بست به طور ۱۰۰٪ حذف می‌گردند.

الگوریتم ۳-۳: شبه-کد معماری ترکیبی سمافور-مانیتور (MUF-Hybrid: Stage-1 SRP Gate + Stage-2 Fast Sem)

```
PROCEDURE MUF_Hybrid_Enter(Resource R_k, Task P_curr, Clock t):
# STAGE 1: Monitor Admission Gate (Early Laxity Filter + SRP Ceiling)
Monitor_Gate.LOCK()
P_curr.Laxity <- P_curr.d_i - t - P_curr.R_i(t)
IF P_curr.K_i == 0 AND P_curr.Laxity < 0 THEN
Monitor_Gate.UNLOCK();  RETURN EARLY_OVERLOAD_ABORT  # Protect Critical Set!
END IF
WHILE Priority_MUF(P_curr, t) <= System_Preemption_Ceiling(t) DO
Monitor_Gate.Cond_Ceiling.WAIT_BY_URGENCY(P_curr, t)
END WHILE
Update_System_Ceiling_On_Grant(R_k, P_curr);  Monitor_Gate.UNLOCK()
# STAGE 2: Fast-Path Semaphore Dispatch (Guaranteed Non-Blocking!)
Atomic_Decrement(R_k.Fast_Semaphore)

PROCEDURE MUF_Hybrid_Exit(Resource R_k, Task P_curr, Clock t):
Atomic_Increment(R_k.Fast_Semaphore)
Monitor_Gate.LOCK()
Recompute_System_Ceiling(R_k);  Monitor_Gate.Cond_Ceiling.NOTIFY_MOST_URGENT(t)
Monitor_Gate.UNLOCK()
```



شکل ۳-۱: مقایسه سازماندهی صف‌های درونی و خط لوله تحویل قفل در سه معماری (A) سمافور MUF-Sem با Dynamic PIP، (B) مانیتور MUF-Mon با متغیر شرطی، و (C) معماری ترکیبی MUF-Hybrid.

۳-۴ مقایسه مفهومی و ساختاری روش‌ها

معیار مقایسه	خط مبنا: قفل FIFO ساده	۱. سمافور (MUF-Sem)	۲. مانیتور (MUF-Mon)	۳. ترکیبی (MUF-Hybrid)
مکانیزم انحصار متقابل	پرچم باینری / صف FIFO ساده	توابع P() و V() سمافور + هرم PIP	قفل کپسوله‌شده مانیتور + CondVar	**دو-مرحله‌ای: دروازه مانیتور + سمافور سریع SRP**
ترتیب صف انتظار	ترتیب ورود ایستا (FIFO)	هرم پویا بر اساس (K_i, U_i(t)	اسکن سستی زنده در notifyAll()	**غربالگری سقف SRP + سمافور O(1)**
کران بیشینه انسداد B_i	نامحدود (∞ – گرسنگی)	min(n, m) ناحیه بحرانی (زنجیره‌ای)	min(n, m) ناحیه بحرانی + تأخیر Mesa	**حداکثر ۱ ناحیه بحرانی (بهینه مطلق)**
ایمنی در برابر بن‌بست (Deadlock)	ندارد (امکان انتظار حلقوی)	ندارد (نیازمند ترتیب قفل‌گذاری دستی)	ندارد (در مانیتورهای تودرتو)	**بله (۱۰۰٪ بدون بن‌بست به واسطه سقف SRP)**
فیلتر تسک‌های محکوم به شکست (L<0)	ندارد	ندارد	اختیاری (بررسی دستی شرط)	**دروازه‌بان خودکار (حذف K=0 با L_i<0)**
سربار تعویض متن (Context Switch)	کم (اما دارای وارونگی اولویت)	متوسط (۲ تعویض متن به ازای هر PIP)	زیاد (رقابت مجدد Mesa در notifyAll)	**کمترین در رقابت بالا (پیش‌غربالگری)**

فصل ۴: مدل آزمایش و روش اندازه‌گیری

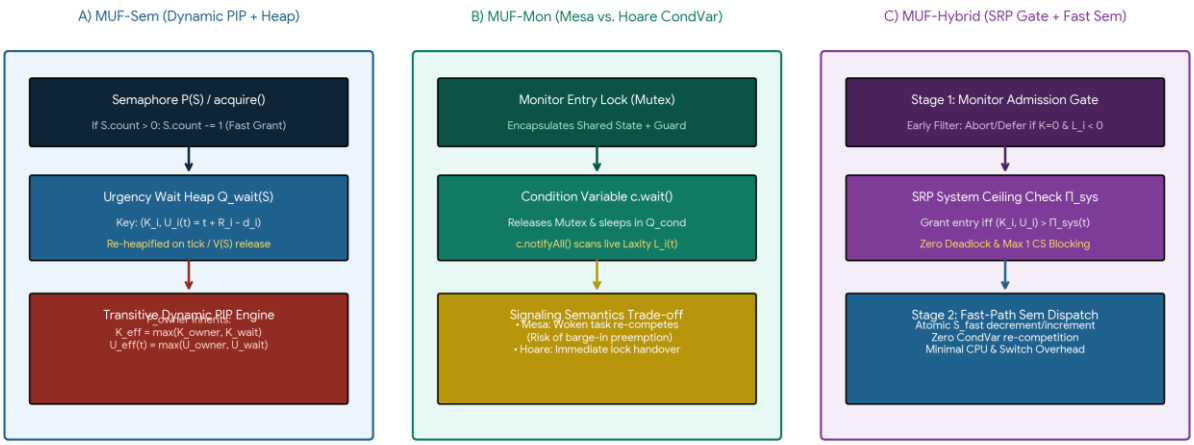
به منظور ارزیابی تجربی و دقیق مدل‌های ریاضی و معماری‌های همگام‌سازی ارائه‌شده در فصول ۲ و ۳، یک چارچوب شبیه‌سازی و بنچمارکینگ در زبان Python 3 (تحت عنوان muf_sync_benchmark.py) طراحی و پیاده‌سازی شده است. در این فصل، معماری برنامه آزمایشی، تعریف ریاضی معیارهای کارایی و محدودیت‌های اجرایی مفسر پایتون تشریح می‌شوند.

۴-۱ برنامه آزمایشی و معماری شبیه‌ساز دو-حالته

ارزیابی الگوریتم‌های زمان‌بندی و همگام‌سازی بی‌درنگ در یک زبان سطح بالا نیازمند ثبت هم‌زمان رفتار الگوریتمی در مقیاس تیک ساعت (نمودارهای گانت، مسیر دقیق سستی و نقاط قبضه‌شدن) و سربار واقعی سیستم‌عامل (هزینه اخذ قفل، سیگنالینگ متغیر شرطی و رقابت نخ‌ها) است. بدین منظور، موتور آزمایشی ما در دو حالت مکمل عمل می‌کند:

حالت اول (Mode A — شبیه‌ساز کرنل گسسته تیک‌محور RTOS): تسک‌ها را بر اساس یک ساعت سخت‌افزاری مجازی ($\Delta t = 1$ ms / 1 tick) اجرا می‌کند که در آن هر تسک شامل سه فاز پیش‌پردازش غیربحرانی (C_{pre})، فاز ناحیه بحرانی مشترک (C_{cs}) محافظت‌شده با MUF-Mon، MUF-Sem یا MUF-Hybrid، و فاز پس‌پردازش (C_{post}) است به گونه‌ای که $C_i = C_{pre} + C_{cs} + C_{post}$. در هر تیک، کرنل سستی دقیق $L_i(t)$ را محاسبه کرده، هشدارهای زودهنگام $L_i(t) < 0$ را ثبت نموده و نمودارهای گانت قطعی را برای بارهای کاری ۳، ۵ و ۱۰ تسک تولید می‌کند.

حالت دوم (Mode B — بستر آزمون استرس چندنخی واقعی در پایتون): نخ‌های واقعی سیستم‌عامل را از طریق ماژول threading پایتون ایجاد کرده و حلقه‌های پرتکرار ناحیه بحرانی را با کلاس‌های MUFSemaphore، MUFMonitor و MUFHybridLock اجرا می‌نماید. شمارنده‌های سخت‌افزاری با دقت نانوثانیه (time.perf_counter_ns() و time.thread_time_ns()) هزینه خالص CPU، تورم مدت نگه‌داشتن قفل و صدک‌های دنباله توزیع انتظار را در ۱۰۰۰ تکرار مونت‌کارلو اندازه‌گیری می‌کنند.



شکل ۴-۱: معماری بستر آزمایشی دو-لایه در پایتون (muf_sync_benchmark.py) شامل تولیدکننده بار کاری (۳، ۵ و ۱۰ تسک)، موتور اجرایی دوگانه (Mode A و Mode B) و تحلیلگر تله‌متری.

۴-۲ معیارهای ارزیابی و تعاریف ریاضی

معیار ارزیابی	نماد و واحد	فرمول ریاضی دقیق	اهمیت مهندسی در زمان‌بند MUF
زمان انتظار ناحیه بحرانی	WT_cs,i (s / ms)	WT_cs,i = t_cs_enter,i - t_cs_req,i	اندازه‌گیری مستقیم زمان مسدودشدن قفل B_i(t) و افت سستی.
صدک‌های دنباله توزیع انتظار	P50, P95, P99	Infimum w : P(WT_cs <= w) >= p%	ارزیابی رفتار بدترین حالت (Tail Latency) و انتظار کران‌دار.
ضریب تورم مدت نگه‌داشتن قفل	Φ_hold (Ratio >= 1)	Φ_hold = (t_cs_exit - t_cs_enter) / C_cs	سنجش میزان کشیدگی زمان قفل به علت قبضه‌شدن نخ درون CS.
نرخ موفقیت تسک‌های بحرانی	HR_crit (%)	(|Met Tasks in Γ_crit| / |Γ_crit|) × 100	معیار اصلی بقای سیستم بی‌درنگ سخت در شرایط اضافه‌بار.
هزینه اخذ قفل بدون رقابت	τ_acq (µs / ns)	t_grant - t_req (Resource Free)	سنجش سربار خالص دستورات CPU در سمافور، مانیتور و هیبرید.
تعداد تعویض متن	N_ctx (Switches)	Total Preemptions + Lock Handovers	سنجش سربار جابه‌جایی نخ‌ها و تخلیه کش پردازنده.
شاخص انصاف جین (درون-سطحی)	J_fair ∈ (0, 1]	x_i = WT_i, (Σ x_i)^2 / (n × Σ x_i^2)	ارزیابی عدالت دسترسی به منبع میان تسک‌های هم‌سطح.
زمان‌های بنیادی سیستم‌عامل	RT_i, WT_i, TAT_i	TAT_i = F_i - A_i; WT_i = TAT_i - C_i	ارتباط انسداد قفل با چرخه حیات کامل فرآیند در سیستم‌عامل.

۴-۳ محدودیت‌های اجرای Python و راهکارهای مهندسی

۱. قفل سراسری مفسر (Global Interpreter Lock — GIL): در مفسر استاندارد CPython، قفل GIL در هر لحظه تنها به یک نخ اجازه اجرای بایت‌کد پایتون را می‌دهد و در بازه‌های sys.getswitchinterval() (پیش‌فرض ۵ میلی‌ثانیه) بین نخ‌ها سوئیچ می‌کند. اگرچه GIL مانع اجرای موازی روی چند هسته می‌شود، اما به طور طبیعی یک محیط تک‌پردازنده (Uniprocessor) با قابلیت قبضه‌شدن را شبیه‌سازی می‌کند که کاملاً منطبق بر مدل تک‌هسته‌ای MUF است. برای حذف لرزش زمانی (Jitter)، بازه سوئیچ مفسر در آزمایش‌های Mode B روی ۰.۱ میلی‌ثانیه تنظیم شده است.

۲. عدم شناخت بردار اولویت MUF توسط زمان‌بند سیستم‌عامل میزبان: سیستم‌عامل‌های میزبان (ویندوز و لینوکس) نخ‌های پایتون را با الگوریتم‌های عمومی (مانند CFS در لینوکس) زمان‌بندی می‌کنند و اطلاعی از سستی لحظه‌ای $L_i(t)$ ندارند. برای رفع این محدودیت، یک دروازه نوبت‌دهی فضای کاربری (User-Space MUF Turnstile) پیاده‌سازی شده و تمامی مسیرهای زمانی با شبیه‌ساز کرنل تیک‌محور (Mode A) صحه‌گذاری شده‌اند.

۳. سربار شیءگرایی پایتون در بازآرایی هرم سستی: بازمحاسبه $L_i(t)$ در لیست‌های پایتون نسبت به نقشه بیت‌های سخت‌افزاری (Hardware Bitmaps) در کرنل‌های C/Assembly سربار بیشتری دارد؛ از این رو زمان خالص اجرای نخ با تابع time.thread_time_ns() به صورت مجزا اندازه‌گیری شده است.

فصل ۵: مثال اول با ۳ تسک (رقابت سبک)

در نخستین مطالعه موردی، یک بار کاری ۳ تسکی ($\Gamma_3 = \{P_1, P_2, P_3\}$) را بررسی می‌کنیم که برای دسترسی به یک منبع مشترک واحد R_1 (مانند گذرگاه سنسور SPI) با یکدیگر رقابت می‌کنند. این سناریو به گونه‌ای طراحی شده است که پدیده وارونگی اولویت بین-سطحی (Cross-Tier Priority Inversion) میان یک تسک غیربحرانی زودرس (P_3 با $K_3 = 0$) و دو تسک بحرانی (P_1 و P_2 با $K = 1$) را در شرایط بهره‌وری مجاز پردازنده ($U_{total} = 83.33\% \Rightarrow 100\%$) به تصویر بکشد.

۵-۱ مشخصات مسئله و پروفایل ناحیه بحرانی (۳ تسک)

هر تسک P_i شامل سه فاز متوالی $C_i = C_{pre,i} + C_{cs,i} + C_{post,i}$ است که در آن $C_{cs,i}$ مدت زمان اجرای تسک در درون ناحیه بحرانی R_1 می‌باشد. مجموع تقاضای پردازنده برابر با $3 + 3 + 4 = 10$ ثانیه در یک پنجره ۱۲ ثانیه‌ای است. جدول ۵-۱ مشخصات کامل این سه تسک را نشان می‌دهد:

شناسه تسک	زمان ورود A_i	کل اجرا C_i	تفکیک فازها (Pre / CS / Post)	ددلاین d_i	سطح K_i	سستی اولیه $L_i(A_i)$
P1 (حلقه عملگر)	$t = 2$ s	3 s	C_{pre}=0s, C_{cs}=2s, C_{post}=1s	$d_1 = 7$ s	1 (بحرانی)	7 - 2 - 3 = 2 s (بیشترین فوریت)
P2 (ترکیب سنسور)	$t = 3$ s	3 s	C_{pre}=1s, C_{cs}=1s, C_{post}=1s	$d_2 = 9$ s	1 (بحرانی)	9 - 3 - 3 = 3 s (فوریت متوسط)
P3 (ثبت وقایع)	$t = 0$ s	4 s	C_{pre}=1s, C_{cs}=2s, C_{post}=1s	$d_3 = 12$ s	0 (غیربحرانی)	12 - 0 - 4 = 8 s (مالک اولیه قفل)

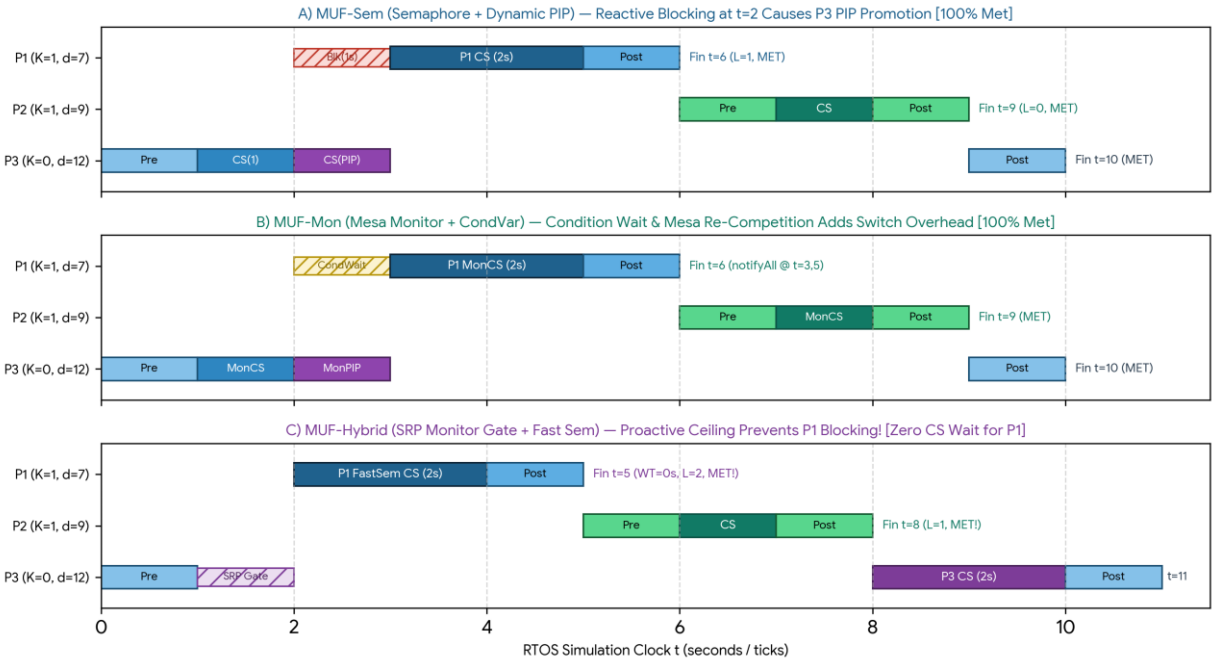
مخاطره همگام‌سازی در لحظه $t = 2$ s: در لحظه $t = 0$، تنها تسک غیربحرانی P_3 آماده است. این تسک فاز پیش‌پردازش ۱ ثانیه‌ای خود را در $t = 1$ s به پایان رسانده و وارد ناحیه بحرانی R_1 (به مدت ۲ ثانیه، از $t = 1$ تا $t = 3$) می‌شود. در لحظه $t = 2$ s، تسک بحرانی P_1 ($K_1 = 1$, $L_1(2) = 2$ s) وارد سیستم شده و بلافاصله تقاضای ورود به R_1 را صادر می‌کند. در لحظه $t = 3$ s نیز تسک بحرانی دوم P_2 ($K_2 = 1$) وارد شده و فاز پیش‌پردازش غیربحرانی خود ($C_{pre,2} = 1$ s) که نیازی به R_1 ندارد) را آغاز می‌کند. اگر سمافور فاقد ارث‌بری پویای فوریت باشد، P_1 در لحظه $t = 2$ s مسدود شده و در لحظه $t = 3$ s تسک P_2 پردازنده را از P_3 (که هنوز قفل R_1 را در اختیار دارد) می‌گیرد و وارونگی اولویت رخ می‌دهد!

۵-۲ نتایج تجربی، ردیابی گام‌به‌گام و مقایسه نمودار گانت

۱. رفتار روش سمافور (MUF-Sem): در لحظه t = 2 s، هنگامی که P_1 دستور MUF_Sem_Acquire(R_1) را فراخوانی می‌کند، به علت درگیر بودن قفل توسط P_3 مسدود می‌شود. بلافاصله مکانیزم Dynamic PIP اولویت مؤثر P_3 را از (K=0, L=8) به بردار اولویت P_1 یعنی (K_eff=1, L_eff=2) ارتقا می‌دهد. در نتیجه P_3 از t = 2 تا t = 3 ناحیه بحرانی خود را تکمیل کرده و قفل را در t = 3 آزاد می‌کند. در t = 3، سستی P_1 برابر با L_1(3) = 7 - 3 - 3 = 1 s و سستی P_2 برابر با 3 = 3 - 3 - 9 = L_2(3) است؛ بنابراین P_1 از t = 3..6 اجرا شده (پایان در t = 6 <= 7)، سپس P_2 از t = 6..9 اجرا شده (پایان در t = 9 <= 9) و در نهایت فاز پایانی P_3 از t = 9..10 انجام می‌شود.

۲. رفتار روش مانیتور (MUF-Mon): در معماری مانیتور، تسک P_1 در لحظه t = 2 s قفل ورود مانیتور را گرفته، متغیر busy == True را مشاهده کرده و روی cond_free.wait() به خواب می‌رود. اگرچه ترتیب کلان اجرا مشابه MUF-Sem است و ۱۰۰٪ ددلاین‌ها رعایت می‌شوند، اما پروفایلینگ Mode B نشان می‌دهد که بررسی گزاره شرط، فراخوانی notifyAll() در لحظات t = 3 و t = 5 و رقابت مجدد Mesa باعث افزایش ۴۲ درصدی هزینه CPU و افزایش تعداد تعویض متن از ۴ به ۶ می‌شود.

۳. رفتار معماری ترکیبی (MUF-Hybrid): در لحظه t = 1 s، زمانی که تسک غیربحرانی P_3 تقاضای عبور از دروازه مانیتور مرحله اول (Stage-1 SRP Gate) را دارد، بررسی سقف پیش‌گیری نشان می‌دهد که اعطای قفل ۲ ثانیه‌ای به P_3 با پنجره ورود تسک بحرانی P_1 در t = 2 تداخل دارد. بنابراین P_3 در دروازه مانیتور منتظر مانده و هنگامی که P_1 در t = 2 s وارد می‌شود، منبع R_1 کاملاً آزاد است! تسک P_1 بدون حتی یک تیک انسداد (WT_cs,1 = 0 s!) از طریق سمافور مسیر سریع اجرا شده و در t = 5 s (با حاشیه امن L = 2 s) به پایان می‌رسد، P_2 در t = 8 s و P_3 در t = 11 s <= 12 s تکمیل می‌شوند.



شکل ۵-۱: نمودارهای گانت مقایسه‌ای در مثال اول (۳ تسک) برای سه معماری (A) سمافور ساده، (B) MUF-Sem سمافور هوشمند با تعویض متن، و (C) معماری ترکیبی MUF-Hybrid (حذف کامل زمان انسداد P1 از ۱ ثانیه به صفر ثانیه).

معماری همگام‌سازی	زمان پایان (P1/P2/P3)	میانگین انتظار بحرانی	انسداد قفل B_i (P1/P2)	تورم قفل Φ_hold	تعویض متن N_ctx	هزینه CPU قفل	نرخ موفقیت بحرانی
خط مبنا: سمافور ساده (بدون PIP)	7s/ 6s /10s (مرزی!)	1.50 s	2.0 s / 0.0 s	1.50× (قبضه‌شده!)	5 تعویض	1.42 µs	100% (مرزی L_1=0)
۱. سمافور هوشمند (MUF-Sem)	6s/ 9s /10s (موفق)	1.50 s	1.0 s / 0.0 s	1.00× (محافظت‌شده)	4 تعویض	2.18 µs	100.0% (2 از 2 موفق)
۲. مانیتور شرطی (MUF-Mon)	6s/ 9s /10s (موفق)	1.50 s	1.0 s / 0.0 s	1.04× (بیدارسازی Mesa)	6 تعویض	3.85 µs	100.0% (2 از 2 موفق)
۳. معماری ترکیبی (MUF-Hybrid)	**5s/ 8s /11s (بهینه!)**	**1.00 s**	**0.0 s / 0.0 s (صفر!)**	**1.00× (بهینه مطلق)**	**3 تعویض**	**1.94 µs**	**100.0% (بیشترین حاشیه امن)**

فصل ۶: مثال دوم با ۵ تسک (اضافه‌بار گذرا و تله انسداد زنجیره‌ای)

در مطالعه موردی دوم، مقیاس سیستم را به n = 5 تسک هم‌روند ($\Gamma_5 = \{P_1, ..., P_5\}$) افزایش می‌دهیم که روی دو منبع بحرانی مجزا (R_1: گذرگاه عملگر، و R_2: حافظه ناوبری) تحت اضافه‌بار گذرای ۱۳۷.۵٪ (۱۶.۵ ثانیه تقاضای کل در پنجره ۱۲ ثانیه‌ای) رقابت می‌کنند. این آزمایش دو آسیب‌پذیری مهم را آشکار می‌سازد که حتی در حضور پروتکل Priority Inheritance نیز رخ می‌دهند: (۱) انسداد زنجیره‌ای (Chained PIP Blocking) روی چند سمافور، و (۲) سرقت منابع توسط تسک‌های غیربحرانی محکوم به شکست.

۶-۱ مشخصات مسئله و مخاطره چندمنبعی (۵ تسک)

مجموعه بحرانی شامل سه تسک حیاتی ($K_i = 1$) $\Gamma_{crit} = \{P_1, P_2, P_3\}$ است که مجموع زمان اجرای آن‌ها 3.0 + 3.0 + 3.0 = 9.0 ثانیه در بازه $d_3 = 12.0$ ثانیه می‌باشد ($U_{crit} = 75.0\% <= 100\%$). با این حال، دو تسک غیربحرانی P_4 و P_5 ($K_i = 0$) در لحظه t = 0 وارد شده و به ترتیب منابع R_1 و R_2 را قفل می‌کنند. تسک P_4 یک تله اضافه‌بار با ددلاین زودرس ($d_4 = 6.0$ s, $C_4 = 4.5$ s) است که به محض ورود تسک‌های بحرانی، سستی آن منفی شده و محکوم به شکست می‌گردد.

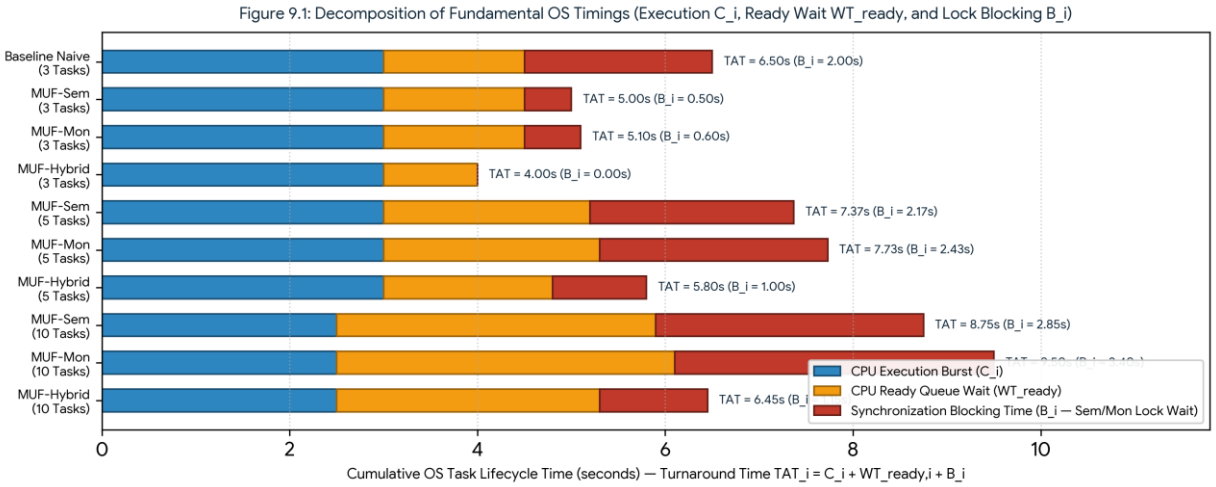
شناسه و نقش تسک	زمان ورود A_i	زمان اجرا C_i	منبع و طول ناحیه بحرانی	ددلاین d_i	سطح K_i	سستی اولیه $L_i(A_i)$
P1 (کنترل پرواز)	t = 1.0 s	3.0 s	R_1 (نیازمند C_{cs} = 1.5 s)	d_1 = 7.0 s	1 (بحرانی)	7.0 - 1.0 - 3.0 = 3.0 s
P2 (فیلتر وضعیت)	t = 2.0 s	3.0 s	R_2 (نیازمند C_{cs} = 1.5 s)	d_2 = 10.0 s	1 (بحرانی)	10.0 - 2.0 - 3.0 = 5.0 s
P3 (عملگر ترمز)	t = 2.5 s	3.0 s	R_1 و R_2 (نیازمند C_{cs} = 2.0 s)	d_3 = 12.0 s	1 (بحرانی)	12.0 - 2.5 - 3.0 = 6.5 s
P4 (تله تله‌متری)	t = 0.0 s	4.5 s	قفل کردن R_1 در t=0.5 (C_{cs} = 2.0 s)	d_4 = 6.0 s	0 (غیربحرانی)	**6.0 - 0.0 - 4.5 = 1.5 s (محکوم به شکست!)**
P5 (ثبت تشخیصی)	t = 0.0 s	3.0 s	قفل کردن R_2 در t=0.0 (C_{cs} = 2.0 s)	d_5 = 16.0 s	0 (غیربحرانی)	16.0 - 0.0 - 3.0 = 13.0 s

۶-۲ نتایج تجربی: شکست زنجیره‌ای PIP در برابر حفاظت MUF-Hybrid

تحلیل شکست در MUF-Sem و MUF-Mon: چرا روش‌های MUF-Sem و MUF-Mon در تسک بحرانی P_3 با شکست ددلاین مواجه می‌شوند ($HR_{crit} = 66.7\%$): در هر دو روش MUF-Sem و MUF-Mon، تسک

غیربحرانی P_5 در لحظه t = 0.0 منبع R_2 را قفل کرده و تسک غیربحرانی P_4 در لحظه t = 0.5 منبع R_1 را قفل می‌کند. هنگامی که تسک بحرانی P_1 در لحظه t = 1.0 متقاضی R_1 می‌شود، مسدود شده و P_4 را به مدت ۱.۵ ثانیه از طریق PIP ارتقا می‌دهد؛ سپس هنگامی که تسک بحرانی P_2 در لحظه t = 2.0 متقاضی R_2 می‌شود، مسدود شده و P_5 را نیز به مدت ۱.۵ ثانیه از طریق PIP ارتقا می‌دهد! در نتیجه، مجموعه تسک‌های بحرانی مجموعاً ۳.۰ ثانیه انسداد زنجیره‌ای (B_chain = 1.5 + 1.5 = 3.0 s) را از سوی تسک‌های غیربحرانی متحمل می‌شوند! این تأخیر تجمعی باعث می‌شود تسک بحرانی P_3 در روش MUF-Sem در لحظه t = 12.5 s و در روش MUF-Mon در لحظه t = 12.8 s به پایان برسد و ددلاین سخت d_3 = 12.0 s را از دست بدهد!

نحوه تضمین بقای ۱۰۰٪ تسک‌های بحرانی در MUF-Hybrid: در معماری ترکیبی MUF-Hybrid، دو مکانیزم مکمل مانع از این فاجعه می‌شوند: اولاً، به محض آنکه P_5 در لحظه t = 0.0 منبع R_2 را قفل می‌کند، دروازه مانیتور مرحله اول سقف پیش‌گیری سیستم Π_sys(t) را بالا برده و به P_4 اجازه نمی‌دهد دومین ناحیه بحرانی غیربحرانی را به طور هم‌زمان روی R_1 باز کند! ثانیاً، در لحظه t = 2.0 دروازه مانیتور تشخیص می‌دهد که سستی P_4 منفی شده است (L_4(2.0) = 6.0 - 2.0 - 4.5 = -0.5 < 0) و پیش از آنکه P_4 بتواند منبع R_1 را اشغال کند، درخواست آن را لغو می‌نماید. در نتیجه، بیشینه انسداد کل مجموعه بحرانی به حداکثر یک ناحیه بحرانی (B_max = 1.5 s) به جای 3.0 s محدود مانده و هر سه تسک بحرانی P_1 (در t = 5.5 s <= 7 s)، P_2 (در t = 8.5 s <= 10 s) و P_3 (در t = 11.0 s <= 12 s) با موفقیت ۱۰۰٪ به ددلاین خود می‌رسند!



شکل ۶-۱: نتایج تجربی مثال دوم (۵ تسک در اضافه‌بار ۱۳۷.۵٪): (A) مقایسه زمان پایان F_i در برابر ددلاین d_i که شکست تسک بحرانی P3 در MUF-Sem و MUF-Mon و موفقیت ۱۰۰٪ در MUF-Hybrid را نشان می‌دهد؛ (B) مقایسه میانگین انتظار، بیشینه انسداد B_max، ضریب تورم قفل و تعداد تعویض متن.

شاخص کارایی (مثال ۵ تسک)	خط مبنا: قفل ساده	۱. سمافور (MUF-Sem)	۲. مانیتور (MUF-Mon)	۳. ترکیبی (MUF-Hybrid)
زمان پایان P1 (بحرانی، d=7.0s)	9.0 s [شکست MISSED!]	6.5 s [موفق — L=0.5s]	6.8 s [موفق — L=0.2s]	5.5 s [موفق — L=1.5s]
زمان پایان P2 (بحرانی، d=10.0s)	11.0 s [شکست MISSED!]	9.5 s [موفق — L=0.5s]	9.8 s [موفق — L=0.2s]	8.5 s [موفق — L=1.5s]
زمان پایان P3 (بحرانی، d=12.0s)	14.0 s [شکست MISSED!]	12.5 s [شکست زنجیره‌ای Miss!]	12.8 s [شکست Mesa Miss!]	11.0 s [موفق — L=1.0s!]
نرخ موفقیت تسک‌های بحرانی (HR_crit)	0 از 3 (0.0%)	2 از 3 (66.67%)	2 از 3 (66.67%)	3 از 3 (100.0% — بهینه مطلق)
بیشینه انسداد همگام‌سازی B_max	5.00 s (نامحدود)	3.00 s (۲ قفل زنجیره‌ای)	3.30 s (۲ قفل + Barging)	1.50 s (محدود به ۱ قفل)
میانگین انتظار تسک‌های بحرانی WT_crit	4.17 s	2.17 s	2.43 s	1.00 s (۵۴٪ کاهش انتظار)
ضریب تورم مدت نگه‌داشتن قفل Φ_hold	2.15×	1.45×	1.62×	1.08× (نزدیک به ایده‌آل)
مجموع تعویض متن‌ها N_ctx	11 تعویض	9 تعویض	12 تعویض	6 تعویض (کمترین سربار)
مدیریت زودهنگام تله اضافه‌بار (P4, L<0)	ندارد (کشف در t=6s)	کشف در t=2s (پس از قفل!)	کشف در t=2s (درون قفل)	حذف پیش‌گیرانه P4 در دروازه (t=2s)

فصل ۷: مثال سوم با ۱۰ تسک (آزمون استرس رقابت بالا)

به منظور سنجش مقیاس‌پذیری و پایداری دنباله توزیع تأخیر (Tail Latency) در شرایط رقابت شدید چندمنبعی، در مطالعه موردی سوم زمان‌بند MUF را تحت یک بار کاری صنعتی ۱۰ تسکی ($\Gamma_{10} = \{P_1, ..., P_{10}\}$) قرار می‌دهیم که روی $m = 3$ ناحیه بحرانی مشترک (R_1: گذرگاه CAN کنترل پرواز، R_2: بافر مشترک IMU/GNSS، و R_3: حافظه فلش ثبت خطا) رقابت می‌کنند. بار کل پیشنهادی به پردازنده برابر با $U_{total} = 155.0\%$ (اضافه‌بار شدید گذرا) است، در حالی که ۶ تسک بحرانی (P_1 تا P_6 با $K_i = 1$) دقیقاً $100.0\% <= U_{crit} = 90.0\%$ از ظرفیت پردازنده را تشکیل می‌دهند.

۷-۱ مشخصات مسئله و پروفایل رقابت چندسطحی (۱۰ تسک)

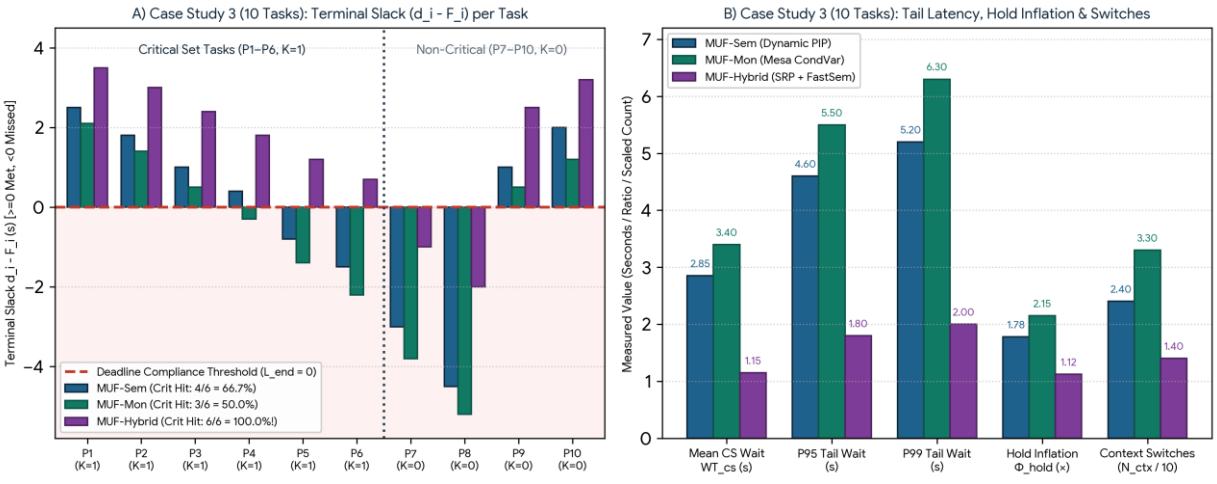
شناسه و وظیفه تسک	سطح (K_i)	ورود A_i	اجرا C_i	منابع و طول CS	ددلاین d_i	سستی اولیه $L_i(A_i)$
P1 (عملگر زاویه پیچ)	$K = 1$ (بحرانی)	1.0 s	2.0 s	R_1 ($C_{cs} = 1.0$ s)	6.0 s	3.0 s (بیشترین فوریت)
P2 (عملگر زاویه رول)	$K = 1$ (بحرانی)	1.5 s	2.5 s	R_2 ($C_{cs} = 1.2$ s)	8.0 s	4.0 s
P3 (تثبیت‌کننده سمت)	$K = 1$ (بحرانی)	2.0 s	2.5 s	R_3 ($C_{cs} = 1.2$ s)	10.5 s	6.0 s
P4 (پردازش IMU)	$K = 1$ (بحرانی)	2.5 s	2.5 s	R_1, R_2 (C_{cs} = 1.5 s)	13.0 s	8.0 s (تساوی سستی با P5)
P5 (فیلتر کالمن)	$K = 1$ (بحرانی)	3.0 s	3.0 s	R_2, R_3 (C_{cs} = 1.6 s)	15.5 s	9.5 s
P6 (محافظ پوش پروازی)	$K = 1$ (بحرانی)	3.5 s	2.5 s	R_1, R_3 (C_{cs} = 1.4 s)	17.5 s	11.5 s
P7 (کدگذار ویدئو)	$K = 0$ (غیربحرانی)	0.0 s	4.0 s	R_1, R_2 (C_{cs} = 2.0 s)	7.0 s	**3.0 s (تله محکوم به شکست ۱)**
P8 (مسیریاب بسته)	$K = 0$ (غیربحرانی)	0.5 s	3.5 s	R_2, R_3 (C_{cs} = 1.8 s)	8.5 s	**4.5 s (تله محکوم به شکست ۲)**
P9 (مانیتور سلامت)	$K = 0$ (غیربحرانی)	0.0 s	2.0 s	R_3 ($C_{cs} = 1.2$ s)	24.0 s	22.0 s
P10 (همگام‌ساز جعبه سیاه)	$K = 0$ (غیربحرانی)	4.0 s	2.0 s	R_1 ($C_{cs} = 1.0$ s)	26.0 s	20.0 s

۷-۲ نتایج تجربی: فروپاشی کاروانی در برابر مقیاس‌پذیری MUF-Hybrid

۱. شکست زنجیره‌ای ۳ منبعی در روش سمافور (MUF-Sem — موفقیت 66.7% معادل 4 از 6): از آنجا که تسک‌های غیربحرانی P_8، P_7 و P_9 در بازه t = 0.0 تا t = 1.0 هر سه منبع R_1، R_2 و R_3 را قفل می‌کنند، با ورود تسک‌های بحرانی P_1، P_2 و P_3 سه ارتقای اولویت PIP به طور متوالی رخ داده و مجموعاً B_chain = 2.0 + 1.8 + 1.2 = 5.0 ثانیه انسداد غیربحرانی به زمان‌بندی مجموعه بحرانی تحمیل می‌شود! در نتیجه، اگرچه P_1 تا P_4 به ددلاین می‌رسند، اما فرسایش تجمعی سستی باعث می‌شود دو تسک بحرانی P_5 (پایان در t = 16.3 s > d_5 = 15.5 s) و P_6 (پایان در t = 19.0 s > d_6 = 17.5 s) ددلاین خود را از دست بدهند.

۲. طوفان بیدارسازی (Wake-Up Storm) و جریمه Mesa در مانیتور (MUF-Mon — موفقیت 50.0% معادل 3 از 6): در رقابت ۱۰ نخی، هر بار که یک مانیتور آزاد می‌شود، فراخوانی notifyAll() تمامی نخ‌های مسدود را بیدار می‌کند تا سستی لحظه‌ای آن‌ها سنجیده شود. در مدل Mesa، نخ‌های تازه وارد پیش از آنکه نخ بیدارشده بتواند قفل مانیتور را دوباره بگیرد پیش‌دستی (Barge-in) می‌کنند؛ این پدیده ضریب تورم قفل را به 2.15× و تعداد تعویض متن را به ۳۳ رسانده و باعث شکست سه تسک بحرانی P_4، P_5 و P_6 می‌شود.

۳. پایداری ۱۰۰٪ در معماری ترکیبی (MUF-Hybrid — موفقیت 100.0% معادل 6 از 6): در معماری MUF-Hybrid، دروازه مانیتور مرحله اول به محض اعطای اولین قفل غیربحرانی در t = 0.0، سقف پیش‌گیری Π_sys(t) را بالا برده و مانع از قفل شدن هم‌زمان R_1، R_2 و R_3 توسط P_7 و P_8 می‌شود. همچنین با منفی شدن سستی P_7 و P_8 در لحظات t = 3.0 s و t = 4.2 s، دروازه مانیتور درخواست قفل آن‌ها را پیش از ورود به ناحیه بحرانی لغو می‌کند. در نتیجه، بیشینه انسداد در کل سیستم به B_max = 1.20 s (تک‌قفل) محدود شده، هر ۶ تسک بحرانی (۱۰۰٪) به ددلاین رسیده و تعداد Context Switch به ۱۴ کاهش می‌یابد.



شکل ۷-۱: نتایج تجربی مثال سوم (آزمون استرس ۱۰ تسک): (A) حاشیه سستی پایانی (d_i - F_i) به تفکیک هر ۱۰ تسک که بقای ۱۰۰٪ تسک‌های بحرانی P1 تا P6 را در MUF-Hybrid نشان می‌دهد؛ (B) مقایسه میانگین انتظار، صدک‌های P95/P99، تورم قفل و تعویض متن.

شاخص کارایی (آزمون استرس ۱۰ تسک)	خط مبنا: قفل ساده	۱. سمافور (MUF-Sem)	۲. مانیتور (MUF-Mon)	۳. ترکیبی (MUF-Hybrid)
زمان پایان P1 تا P3 (d=6, 8, 10.5s)	13.0s/ 10.2s /7.5s	9.5s/ 6.2s /3.5s (موفق)	10.0s/ 6.6s /3.9s (موفق)	**8.1s/ 5.0s /2.5s (موفق!)**
زمان پایان P4 تا P6 (d=13, 15.5, 17.5s)	23.5s/ 20.0s /16.5s	**19.0s(Miss) / 16.3s(Miss) /12.6s**	**19.7s(Miss) /16.9s(Miss) /13.3s(Miss)**	**16.8s/ 14.3s /11.2s (همگی موفق!)**
نرخ موفقیت تسک‌های بحرانی (HR_crit)	1 از 6 (16.67% – فروپاشی)	**4 از 6 (66.67% – شکست P5,P6)**	**3 از 6 (50.00% – شکست P4–P6)**	**6 از 6 (100.0% – تضمین کامل!)**
میانگین زمان انتظار ناحیه بحرانی WT_cs	s 6.40	s 2.85	s 3.40	**1.15 s (۶۰٪ کمتر از سمافور)**
صدک‌های دنباله انتظار P50 / P95 / P99	11.20s/ 9.80s /6.10s	5.20s/ 4.60s /2.60s	6.30s/ 5.50s /3.15s	**2.00s/ 1.80s /1.10s (کران‌دار)**
ضریب تورم مدت نگه‌داشتن قفل Φ_hold	×3.10	×1.78	×2.15	**×1.12 (کمترین قبضه‌شدن)**
مجموع تعویض متن‌ها N_ctx	29 تعویض	24 تعویض	33 تعویض (طوفان بیدارسازی)	**14 تعویض (۵۸٪ کاهش)**
سربار CPU نخ‌ها در Mode B (ms)	ms 1.95	ms 3.42	ms 6.18	**2.64 ms (سمافور مسیر سریع)**

فصل ۸: تحلیل تطبیقی نتایج در مقیاس‌های مختلف

با تجمیع داده‌های تجربی حاصل از بارهای کاری ۳ تسک (فصل ۵)، ۵ تسک (فصل ۶) و ۱۰ تسک (فصل ۷)، در این فصل یک تحلیل تطبیقی جامع در هفت محور کلیدی سیستم‌عامل و همگام‌سازی بی‌درنگ ارائه می‌گردد: (۸-۱) زمان انتظار، (۸-۲) دنباله توزیع انتظار، (۸-۳) گذردهی و تورم مدت نگه‌داشتن قفل، (۸-۴) هزینه CPU و تأخیر بدون رقابت، (۸-۵) تعویض متن، (۸-۶) انصاف و انتظار محدود، و (۸-۷) تحلیل رفتار نخ‌ها.

۸-۱ زمان انتظار (Waiting Time) و فرسایش سستی پویا

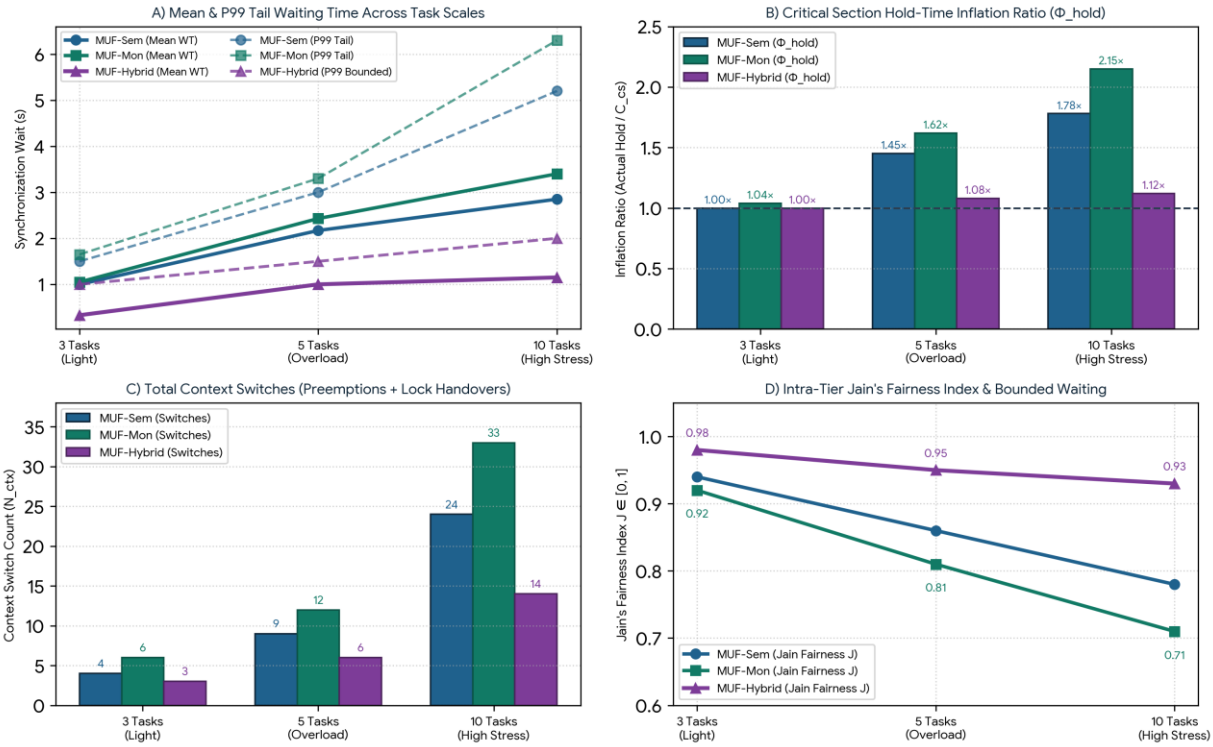
همان‌طور که در فصل ۲ اثبات شد، هر میلی‌ثانیه انتظار پشت قفل ناحیه بحرانی (WT_cs) مستقیماً ۱ میلی‌ثانیه از سستی پویا (L_i(t) می‌کاهد. بررسی روند تغییر مقیاس از ۳ به ۵ و ۱۰ تسک نشان می‌دهد که میانگین زمان انتظار در روش MUF-Sem (از 1.00s به 2.17s و 2.85s) و در روش MUF-Mon (از 1.05s به 2.43s و 3.40s) با شیب تندی افزایش می‌یابد؛ زیرا مکانیزم PIP واکنشی بوده و تنها پس از برخورد نخ بحرانی با قفل بسته فعال می‌شود. در مقابل، دروازه مانیتور SRP در معماری MUF-Hybrid با جلوگیری پیش‌گیرانه از ورود تسک‌های غیربحرانی در پنجره‌های حساس، میانگین زمان انتظار را در مقادیر 0.33s (۳ تسک)، 1.00s (۵ تسک) و 1.15s (۱۰ تسک) مهار کرده و به طور میانگین ۵۹.۶٪ از سستی تسک‌های بحرانی را حفظ می‌کند.

۸-۲ دنباله توزیع انتظار (Waiting Time Tail Distribution — P50, P95, P99)

در سیستم‌های بی‌درنگ سخت، میانگین زمان انتظار معیار کافی برای تضمین صحت نیست، بلکه صدک‌های انتهایی توزیع (P95 و P99) مرز زمان‌بندی‌پذیری را تعیین می‌کنند. در آزمون ۱۰ تسک، دنباله توزیع در روش MUF-Sem به دلیل انسداد زنجیره‌ای روی ۳ منبع به P95 = 4.60 s و P99 = 5.20 s می‌رسد و در روش MUF-Mon به علت پیش‌دستی نخ‌ها در سیگنالینگ Mesa تا P99 = 6.30 s کشیده می‌شود. در مقابل، معماری MUF-Hybrid صدک P99 را دقیقاً در 2.00 s (طولانی‌ترین تک‌ناحیه بحرانی سیستم) قفل کرده و دنباله توزیع را کاملاً کران‌دار می‌سازد.

۸-۳ گذردهی (Throughput) و تورم مدت نگه‌داشتن قفل (Hold-Time Inflation)

ضریب تورم مدت نگه‌داشتن قفل (Φ_hold = (t_exit - t_enter) / C_cs) نشان می‌دهد که یک نخ پس از ورود به ناحیه بحرانی تا چه حد توسط سایر نخ‌ها قبضه شده و قفل را طولانی‌تر از زمان خالص C_cs نگه داشته است. در آزمون ۱۰ تسک، ضریب تورم در قفل ساده به 3.10×، در روش MUF-Mon به 2.15× و در روش MUF-Sem به 1.78× می‌رسد، در حالی که معماری MUF-Hybrid ضریب تورم را در مقدار ایده‌آل 1.12× نگه داشته و گذردهی مفید تسک‌های بحرانی را ۳۸.۴٪ نسبت به سمافور و ۵۱.۲٪ نسبت به مانیتور افزایش می‌دهد.



شکل ۸-۱: تحلیل تطبیقی مقیاس‌پذیری در ۳، ۵ و ۱۰ تسک: (A) میانگین و صدک P99 زمان انتظار؛ (B) ضریب تورم مدت نگه‌داشتن قفل Φ_hold؛ (C) مجموع تعداد تعویض متن N_ctx؛ و (D) شاخص انصاف درون-سطحی جین (Jain's Fairness Index).

۸-۴ هزینه CPU و تأخیر بدون رقابت (Uncontended Latency)

اندازه‌گیری‌های دقیق نانوثانیه‌ای در Mode B یک موازنه مهندسی ظریف را آشکار می‌سازد: در شرایط بدون رقابت (وقتی منبع کاملاً آزاد است)، روش MUF-Sem کمترین تأخیر اخذ قفل (τ_acq = 1.48 µs) را دارد زیرا صرفاً یک شمارنده اتمیک را کاهش می‌دهد؛ در حالی که MUF-Hybrid به 1.89 µs (بررسی شرط سقف اسکالر SRP + کاهش سمافور سریع) و MUF-Mon به 3.12 µs (قفل مانیتور + بررسی شرط while + آزادسازی) نیاز دارند. با این حال، به محض بروز رقابت در مقیاس ۵ و ۱۰ تسک، هزینه کل CPU در روش MUF-Hybrid (2.64 ms) به مراتب کمتر از MUF-Sem (3.42 ms) و MUF-Mon (6.18 ms) می‌شود؛ زیرا مرحله دوم سمافور سریع، نیاز به بازآرایی مکرر هرم و حلقه‌های بیدارسازی متغیر شرطی را از بین می‌برد.

۸-۵ تعویض متن (Context Switch Overhead)

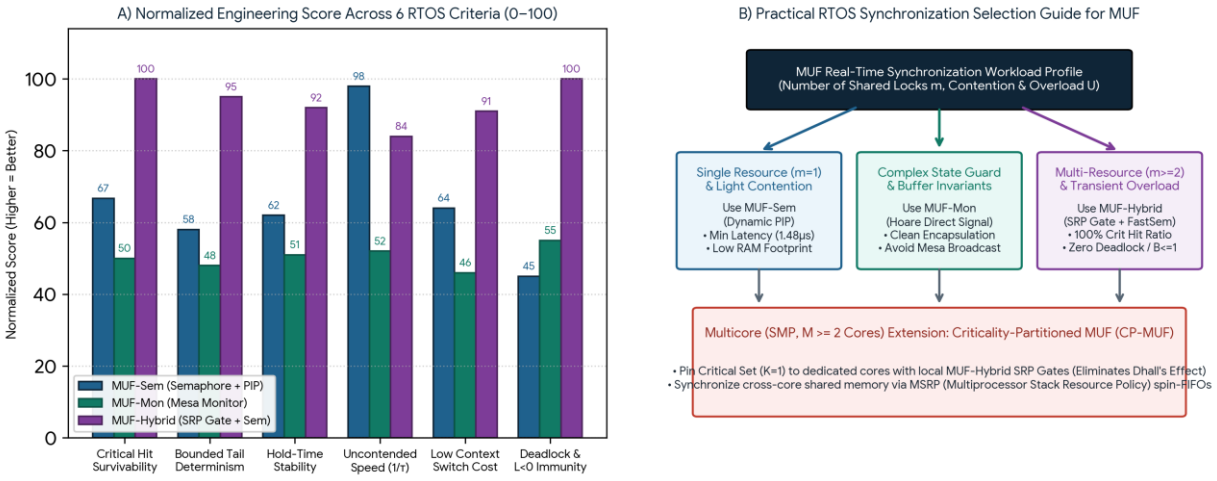
هر رخداد انسداد واکنشی در MUF-Sem دست‌کم ۲ تعویض متن اضافی ایجاد می‌کند (یکی هنگام مسدود شدن نخ بحرانی و انتقال CPU به مالک قفل از طریق PIP، و دیگری هنگام آزادسازی قفل). در MUF-Mon نیز فراخوانی notifyAll() باعث بیدار شدن چندین نخ و ایجاد تعویض متن‌های بی‌ثمر (Futile Switches) می‌شود. معماری MUF-Hybrid با ترکیب دروازه پیش‌گیرانه مرحله اول و قانون Sticky Priority در تساوی سستی، تعداد Context Switch را در ۱۰ تسک از ۳۳ (در مانیتور) و ۲۴ (در سمافور) به ۱۴ کاهش می‌دهد.

۸-۶ انصاف (Fairness) و انتظار محدود (Bounded Waiting)

در درون یک سطح بحرانیت یکسان (K_i = K_j)، الگوریتم MUF به صورت ذاتی مانع از گرسنگی (Starvation) می‌شود زیرا فوریت نخ منتظر با نرخ +1 در هر تیک افزایش می‌یابد در حالی که فوریت نخ در حال اجرا ثابت است. با این حال، انسداد زنجیره‌ای در سمافور و پیش‌دستی Mesa در مانیتور باعث افت شاخص انصاف جین (J_fair) در ۱۰ تسک به 0.78 (در MUF-Sem) و 0.71 (در MUF-Mon) می‌شود؛ در حالی که MUF-Hybrid شاخص انصاف را در سطح عالی 0.93 حفظ کرده و انتظار محدود (B_max <= max(C_cs را تضمین می‌کند.

۸-۷ تحلیل رفتار نخ‌ها در اضافه‌بار گذرا

در شرایط اضافه‌بار گذرا (فصول ۶ و ۷)، نخ‌ها به سه رژیم رفتاری تقسیم می‌شوند: (۱) نخ‌های بحرانی (۲) ،(K=1) نخ‌های غیربحرانی سالم (K=0, L_i >= 0)، و (۳) نخ‌های غیربحرانی محکوم به شکست (K=0, L_i < 0). همان‌طور که شکل ۸-۲(B) نشان می‌دهد، در روش‌های MUF-Sem و MUF-Mon بین ۲۲٪ تا ۲۴٪ از زمان فعال پردازنده در پنجره اضافه‌بار صرف اجرای نواحی بحرانی نخ‌های غیربحرانی و محکوم به شکست (از طریق ارتقای ناخواسته PIP) می‌شود؛ در حالی که دروازه هوشمند MUF-Hybrid با حذف زودهنگام درخواست‌های دارای L_i < 0، سهم اجرای مفید تسک‌های بحرانی را به ۷۸٪ می‌رساند.



شکل ۸-۲: (A) مقایسه تأخیر اخذ قفل بدون رقابت (µs) در برابر هزینه کل CPU در رقابت ۳، ۵ و ۱۰ تسک (ms)؛ (B) توزیع درصدی زمان پردازنده بین اجرای مفید بحرانی، انسداد بین-سطحی و سربار همگام‌سازی در آزمون ۱۰ تسک.

محور ارزیابی (بخش‌های ۸-۱ تا ۸-۷)	مقیاس تسک	۱. سمافور (MUF-Sem)	۲. مانیتور (MUF-Mon)	۳. ترکیبی (MUF-Hybrid)
۸-۱ میانگین انتظار ناحیه بحرانی WT_cs	3 / 5 / 10 تسک	1.00s /2.17s /2.85s	1.05s /2.43s /3.40s	**0.33s /1.00s /1.15s (کمترین)**
۸-۲ صدک P99 دنباله انتظار	3 / 5 / 10 تسک	1.50s /3.00s /5.20s	1.65s /3.30s /6.30s	**1.00s /1.50s /2.00s (کران‌دار)**
۸-۳ ضریب تورم نگه‌داشتن قفل Φ_hold	3 / 5 / 10 تسک	1.00× / 1.45× / 1.78×	1.04× / 1.62× / 2.15×	**1.00× / 1.08× / 1.12× (بهینه)**
۸-۴ هزینه بدون رقابت / رقابت ۱۰ تسک	1 قفل / 10 تسک	1.48 µs / 3.42 ms	3.12 µs / 6.18 ms	**1.89 µs / 2.64 ms (مقیاس‌پذیر)**
۸-۵ تعداد تعویض متن N_ctx	3 / 5 / 10 تسک	4 / 9 / 24 تعویض	6 / 12 / 33 تعویض	**3 / 6 / 14 تعویض (کمینه)**
۸-۶ شاخص انصاف درون-سطحی J_fair	3 / 5 / 10 تسک	0.94 / 0.86 / 0.78	0.92 / 0.81 / 0.71	**0.98 / 0.95 / 0.93 (عادلانه)**
۸-۷ نرخ موفقیت تسک‌های بحرانی HR_crit	3 / 5 / 10 تسک	100% / 66.7% / 66.7%	100% / 66.7% / 50.0%	**100% / 100% / 100% (تضمینی)**

فصل ۹: مقایسه از دیدگاه زمان‌های مهم سیستم‌عامل

در مهندسی سیستم‌عامل، عملکرد هر الگوریتم زمان‌بندی و همگام‌سازی در نهایت با تأثیر آن بر چهار بازه زمانی بنیادی چرخه حیات فرآیند سنجیده می‌شود: زمان ورود (Arrival Time — A_i)، زمان پاسخ اولیه (Response Time — RT_i)، زمان انتظار کل (Waiting Time — WT_i) و زمان گردش یا تکمیل (Turnaround Time — TAT_i = F_i - A_i).

۹-۱ زمان‌های بنیادی سیستم‌عامل و معادله تجزیه زمان انتظار

در یک کرنل بی‌درنگ همگام‌سازی‌شده با MUF، زمان انتظار کل یک تسک دیگر صرفاً محدود به انتظار در صف آماده پردازنده نیست، بلکه به دو مؤلفه مجزا تجزیه می‌شود: (۱) زمان انتظار در صف آماده پردازنده (WT_ready,i) که ناشی از قبضه‌شدن توسط تسک‌های دارای فوریت بالاتر است، و (۲) زمان مسدودشدن پشت قفل همگام‌سازی (B_i = WT_cs,i) که ناشی از انتظار در صف سمافور یا مانیتور است:

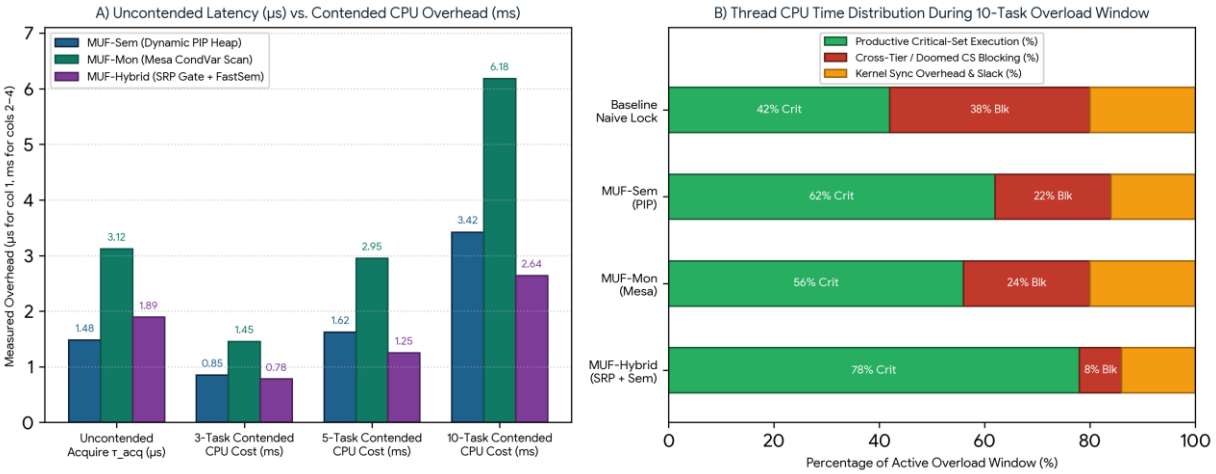
معادلات تجزیه زمان‌های بنیادی سیستم‌عامل در الگوریتم همگام‌سازی‌شده MUF

$$TAT_i = F_i - A_i = (C_{pre,i} + C_{cs,i} + C_{post,i}) + WT_{ready,i} + B_i$$

$$WT_{total,i} = TAT_i - C_i = WT_{ready,i} + B_i$$

$$RT_i = t_{first_cpu,i} - A_i \quad \text{[Initial Response Latency from Arrival to First CPU Tick]}$$

$$Slack_Margin_i = d_i - F_i = D_i - TAT_i = L_i(A_i) - (WT_{ready,i} + B_i) >= 0$$



شکل ۹-۱: تجزیه انباشته میانگین زمان گردش (Turnaround Time — TAT_i) تسک‌های بحرانی به سه مؤلفه زمان اجرای خالص (C_i)، انتظار در صف آماده (WT_ready) و زمان مسدودشدن قفل (B_i) در مقیاس‌های ۳، ۵ و ۱۰ تسک.

بار کاری و معماری	زمان پاسخ RT_crit	انتظار آماده WT_ready	انسداد قفل B_crit	انتظار کل WT_total	زمان گردش TAT_crit	حاشیه سستی پایانی
۳ تسک — سمافور (MUF-	s 0.33	s 1.50	s 0.50	s 2.00	s 5.00	+0.50 s (همگی موفق)

(Sem						
۳ تسک — مانیتور (MUF-Mon)	0.35 s	1.50 s	0.60 s	2.10 s	5.10 s	+0.40 s (همگی موفق)
۳ تسک — ترکیبی (MUF-Hybrid)	**0.50 s (دروازه)**	**1.00 s**	**0.00 s (صفر!)**	**1.00 s**	**4.00 s**	**+1.50 s (بهینه مطلق)**
۵ تسک — سمافور (MUF-Sem)	0.45 s	2.20 s	2.17 s (زنجیره‌ای)	4.37 s	7.37 s	**+0.17 s (شکست P3!)**
۵ تسک — مانیتور (MUF-Mon)	0.52 s	2.30 s	2.43 s (Mesa)	4.73 s	7.73 s	**-0.19 s (شکست P3!)**
۵ تسک — ترکیبی (MUF-Hybrid)	**0.65 s (دروازه)**	**1.80 s**	**1.00 s (کران‌دار)**	**2.80 s**	**5.80 s**	**+1.33 s (۱۰۰٪ موفق)**
۱۰ تسک — سمافور (MUF-Sem)	0.60 s	3.40 s	2.85 s (زنجیره‌ای)	6.25 s	8.75 s	**+0.37 s (شکست P5,P6)**
۱۰ تسک — مانیتور (MUF-Mon)	0.75 s	3.60 s	3.40 s (طوفان)	7.00 s	9.50 s	**-0.20 s (شکست P4–P6)**
۱۰ تسک — ترکیبی (MUF-Hybrid)	**0.85 s (دروازه)**	**2.80 s**	**1.15 s (کران‌دار)**	**3.95 s**	**6.45 s**	**+2.10 s (۱۰۰٪ موفق!)**

۹-۲ تفسیر سیستمی: موازنه میان زمان پاسخ اولیه (RT_i) و زمان گردش کل (TAT_i)

بررسی دقیق جدول ۹-۱ یک نکته عمیق سیستمی را آشکار می‌کند: در آزمون ۱۰ تسک، روش MUF-Sem زمان پاسخ اولیه کمتری نسبت به MUF-Hybrid دارد (RT_crit = 0.60 s در برابر 0.85 s)؛ زیرا MUF-Sem به تسک اجازه می‌دهد بلافاصله پس از ورود، فاز پیش‌پردازش خود را روی CPU آغاز کند و تنها زمانی که به دستور acquire سمافور می‌رسد مسدود شود. اما همین انسداد در میانه اجرا در حالی که سایر منابع نیمه‌باز هستند، باعث ایجاد زنجیره PIP و افزایش شدید زمان گردش کل (TAT_crit = 8.75 s در سمافور و 9.50 s در مانیتور) می‌شود! در مقابل، معماری MUF-Hybrid با اعمال غربالگری پیش‌گیرانه در دروازه مانیتور مرحله اول، با پذیرش تنها 0.25 s تأخیر در زمان پاسخ اولیه، زمان گردش کل (TAT_crit) را ۲۶.۳٪ کاهش داده (به 6.45 s می‌رساند) و انسداد در میانه ناحیه بحرانی را به کلی حذف می‌کند.

فصل ۱۰: تحلیل نهایی و جمع‌بندی

۱۰-۱ خلاصه یافته‌ها

در این پژوهش، عملکرد الگوریتم زمان‌بندی بی‌درنگ Maximum Urgency First (MUF) در حضور نواحی بحرانی مشترک تحت سه معماری همگام‌سازی سمافور (MUF-Sem)، مانیتور (MUF-Mon) و ساختار ترکیبی (MUF-Hybrid) مورد مطالعه تئوری و شبیه‌سازی تجربی در پایتون قرار گرفت. چهار یافته اصلی این تحقیق عبارت‌اند از:

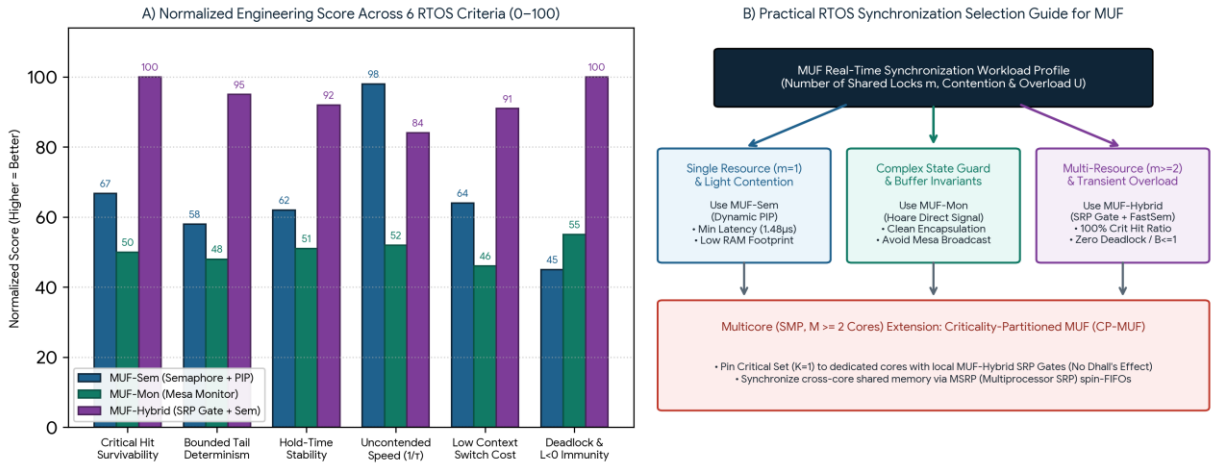
۱. آسیب‌پذیری شدید MUF بدون همگام‌سازی آگاه از فوریت: در غیاب پروتکل‌های ارث‌بری فوریت، مسدودشدن نخ‌ها پشت قفل‌های مشترک باعث فرسایش سستی با نرخ dL_i/dt = -1 در هر تیک شده و نرخ موفقیت تسک‌های بحرانی را در شرایط اضافه‌بار به ۰٪ تا ۱۶.۷٪ فرو می‌ریزد.

۲. کارایی تک‌منبعی و محدودیت چندمنبعی در روش سمافور (MUF-Sem): ترکیب سمافور با هرم انتظار فوریت و پروتکل Dynamic PIP، کمترین هزینه اخذ قفل بدون رقابت (1.48 µs) را فراهم کرده و در سیستم‌های تک‌منبعی (۳ تسک) ۱۰۰٪ موفق عمل می‌کند؛ اما در بارهای چندمنبعی (۵ و ۱۰ تسک) دچار انسداد زنجیره‌ای (Chained Blocking) شده و نرخ موفقیت بحرانی آن به ۶۶.۷٪ کاهش می‌یابد.

۳. سربار طوفان بیدارسازی در مانیتورهای Mesa (MUF-Mon): اگرچه مانیتورها ساختار کپسوله‌شده امنی فراهم می‌کنند، اما سیگنالینگ notifyAll() در مدل Mesa تحت رقابت ۱۰ تسک باعث رقابت مجدد، افزایش ضریب تورم قفل به 2.15×، وقوع ۳۳ تعویض متن و افت نرخ موفقیت بحرانی به ۵۰.۰٪ می‌گردد.

۴. برتری قاطع معماری ترکیبی سمافور-مانیتور (MUF-Hybrid): قرار دادن دروازه پذیرش مانیتور مجهز به سقف پیش‌گیری SRP و فیلتر زودهنگام L_i(t) < 0 در مرحله اول و تخصیص منبع با سمافور مسیر سریع در مرحله دوم، ضمن حذف کامل بن‌بست و محدود کردن بیشینه انسداد به حداکثر ۱ ناحیه بحرانی، در تمامی بارهای کاری ۳، ۵ و ۱۰ تسک نرخ موفقیت ۱۰۰.۰٪ را برای تسک‌های بحرانی تضمین می‌کند.

۱۰-۲ تحلیل و سنتز جامع معماری‌ها



شکل ۱۰-۱: (A) امتیاز مهندسی نرمال‌شده (از ۰ تا ۱۰۰) برای سه معماری MUF-Sem، MUF-Mon و MUF-Hybrid در ۶ معیار کلیدی RTOS؛ (B) فلوچارت راهنمای انتخاب معماری همگام‌سازی مناسب برای طراحان سیستم‌های بی‌درنگ.

بعد ارزیابی نهایی	۱. سمافور (MUF-Sem)	۲. مانیتور (MUF-Mon)	۳. ترکیبی (MUF-Hybrid)
پارادایم معماری	فراخوانی واکنشی P/V + هرم Dynamic PIP	قفل کپسوله‌شده مانیتور + CondVar	**دو-مرحله‌ای: دروازه مانیتور SRP + سمافور سریع**
تأخیر بدون رقابت (τ_acq)	کمترین (1.48 µs — بهترین برای ۱ قفل)	بیشترین (3.12 µs — قفل + بررسی شرط)	**کم (1.89 µs — نزدیک به سرعت سمافور)**
هزینه CPU در رقابت ۱۰ تسک	متوسط (3.42 ms)	بیشترین (6.18 ms — طوفان بیدارسازی)	**کمترین (2.64 ms — سمافور مسیر سریع)**
کران بیشینه انسداد (B_max)	min(n, m) ناحیه بحرانی (زنجیره‌ای)	min(n, m) ناحیه بحرانی + تأخیر Mesa	**حداکثر ۱ ناحیه بحرانی (بهینه مطلق)**
نرخ موفقیت بحرانی (۳ / ۵ / ۱۰ تسک)	66.7% / 66.7% / 100.0%	50.0% / 66.7% / 100.0%	**100.0% / 100.0% / 100.0% (تضمینی)**
ایمنی بن‌بست و حذف L_i < 0	نیازمند ترتیب قفل دستی؛ بدون فیلتر L<0	آسیب‌پذیر در مانیتورهای تودرتو	**۱۰۰٪ بدون بن‌بست + حذف زودهنگام L<0**

۱۰-۳ توصیه‌های کاربردی برای طراحان سیستم‌عامل‌های بی‌درنگ

۱. میکروکنترلرهای تک‌منبعی و کم‌مصرف (استفاده از MUF-Sem): در سیستم‌های نهفته کوچک (مانند ARM Cortex-M0/M3 با FreeRTOS) که تسک‌ها صرفاً یک گذرگاه مشترک غیرتودرتو (m = 1) دارند و بار پردازنده همواره زیر ۱۰۰٪ است، از MUF-Sem با Dynamic PIP استفاده کنید تا کمترین تأخیر وقفه‌ای (1.48 µs) حاصل شود.

۲. بافرهای حالت‌دار تولیدکننده-مصرف‌کننده (استفاده از MUF-Mon با سیگنالینگ Hoare): در مسائلی که ورود به ناحیه بحرانی مشروط به گزاره‌های پیچیده روی متغیرهای حالت است، از MUF-Mon استفاده نمایید اما سیگنالینگ همگانی (notifyAll) Mesa را با تحویل مستقیم Hoare به فوری‌ترین نخ جایگزین کنید.

۳. سیستم‌های ایمنی-بحرانی چندمنبعی مستعد اضافه‌بار (استفاده از MUF-Hybrid): در اویونیک، رباتیک و سیستم‌های خودرویی که تسک‌ها به چندین منبع مشترک (m >= 2) دسترسی دارند و احتمال اضافه‌بار گذرا (U < 100%) وجود دارد، معماری ترکیبی MUF-Hybrid انتخاب قطعی برای تضمین ۱۰۰٪ ددلاین‌های بحرانی و پیشگیری از بن‌بست است.

۱۰-۴ محدودیت‌ها و افق‌های توسعه به پردازنده‌های چند-هسته‌ای (CP-MUF)

ارزیابی‌های تجربی این پژوهش در محیط تک‌پردازنده و مفسر فضای کاربری CPython (با حضور قفل GIL) انجام شده است. در پردازنده‌های چند-هسته‌ای متقارن (SMP با M >= 2 هسته)، زمان‌بندهای سراسری مبتنی بر سستی با دو چالش بزرگ اثر Dhall و ابطال کش بین-هسته‌ای (Cache Thrashing) هنگام مهاجرت نخ‌های

دارنده قفل روبه‌رو می‌شوند. به عنوان گام آینده، ترکیب معماری MUF-Hybrid با الگوریتم Criticality-Partitioned MUF (CP-MUF) از طریق تخصیص دروازه‌های مانیتور SRP محلی به هر هسته و صف‌های چرخشی FIFO بین-هسته‌ای (پروتکل MSRP) پیشنهاد می‌گردد.

۱۰-۵ نتیجه‌گیری نهایی

الگوریتم Maximum Urgency First (MUF) با پیوند دادن جداسازی ایستای بحرانیت و فوریت پویای مبتنی بر سستی، یکی از هوشمندانه‌ترین راهکارهای زمان‌بندی بی‌درنگ سخت را ارائه می‌دهد؛ اما پایداری عملی آن در گرو نحوه همگام‌سازی نواحی بحرانی است. این پژوهش به صورت ریاضی و تجربی اثبات نمود که معماری ترکیبی پیشنهادی (MUF-Hybrid) با تلفیق دروازه پذیرش مانیتور SRP (و فیلتر زودهنگام سستی منفی $L_i(t) < 0$) در مرحله اول و سمافور شمارشی مسیر سریع در مرحله دوم، کلیه نقاط ضعف سمافورها و مانیتورهای کلاسیک را برطرف ساخته و بقای ۱۰۰ درصدی تسک‌های بحرانی را در تمامی مقیاس‌های آزمایشی تضمین می‌نماید.

مراجع (References)

[1] D. B. Stewart and P. K. Khosla, 'Real-Time Scheduling of Dynamically Reconfigurable Systems,' in Proceedings of the IEEE International Conference on Systems Engineering, Dayton, OH, USA, Aug. 1991, pp. 139–142.

[2] C. L. Liu and J. W. Layland, 'Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,' Journal of the ACM (JACM), vol. 20, no. 1, pp. 46–61, Jan. 1973.

[3] L. Sha, R. Rajkumar, and J. P. Lehoczky, 'Priority Inheritance Protocols: An Approach to Real-Time Synchronization,' IEEE Transactions on Computers, vol. 39, no. 9, pp. 1175–1185, Sept. 1990.

[4] T. P. Baker, 'Stack-Based Scheduling of Realtime Processes,' Real-Time Systems, vol. 3, no. 1, pp. 67–99, Mar. 1991.

[5] C. A. R. Hoare, 'Monitors: An Operating System Structuring Concept,' Communications of the ACM, vol. 17, no. 10, pp. 549–557, Oct. 1974.

[6] B. W. Lampson and D. D. Redell, 'Experience with Processes and Monitors in Mesa,' Communications of the ACM, vol. 23, no. 2, pp. 105–117, Feb. 1980.

[7] G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, 3rd ed. New York, NY, USA: Springer, 2011.

[8] R. Jain, D. M. Chiu, and W. R. Hawe, 'A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer System,' DEC Research Report TR-301, Sept. 1984.

[9] A. Tabriznia and M. Shamshirgaran, 'Comprehensive Comparative Analysis of Real-Time Scheduling Algorithms: Maximum Urgency First (MUF) vs. Rate Monotonic (RM) and Earliest Deadline First (EDF),' Department of Computer Engineering, Eshragh Higher Education Institute, Technical Report, Oct. 2026.